IoT Based Smart Robotic Assistance for Alzheimer's Patient



Final Year Project Design Report

Presented By

Zeenat Fatima
22-CE-002

M.Haris Ikram
22-CE-015

Noor uz Zaman
22-CE-027

Supervisor: Dr. Sobia Arshad

Department of Computer Engineering

HITEC University, Taxila

Session 2022-2026



Department of Computer Engineering
HITEC University, Taxila

Project Title

IoT Based Smart Robotic Assistance for Alzheimer's Patient

Presented by

Names	Registration No
Zeenat Fatima	22-CE-002
M.Haris Ikram	22-CE-015
Noor uz Zaman	22-CE-027

In partial fulfilment of the four years **B.S. Computer Engineering** requirements, HITEC University Taxila, Pakistan underneath the supervision of its project adviser as well as authorized by the project review board has been recognized.

_____

Dr. Sobia Arshad
Assistant Professor, Project Advisor

_____

Dr. Sohaira Ahmad
Chairman
Department of Computer Engineering

Certificate

It is certified that Zeenat Fatima, M.Haris Ikram, and Noor uz Zaman(22-CE-002/015/027) have carried out all the work related to this project under my guidance and supervision at the Department of Computer Engineering, HITEC University and the work fulfills and meets the prerequisites for the award of **BS Computer Engineering** degree.

Date: _____

Supervisor:

_____

Dr. Sobia Arshad
Assistant Professor
Department of Computer Engineering,
HITEC University, Taxila

Head of Department:

_____

Dr. Sohaira Ahmad
Chairman
Department of Computer Engineering,
HITEC University, Taxila

Declaration

We, hereby declare that this project neither as a whole nor as a part there of has been copied out from any source. It is further declared that we have developed this project and the accompanied report entirely on the basis of our personal efforts made under the sincere guidance of our supervisor. No portion of the work presented in this report has been submitted in the support of any other degree or qualification of this or any other University or Institute of learning. If any violation is found in this FYP report, we will be liable to be punished under the plagiarism rules of HEC.

Signature_____

Name: Zeenat Fatima

Signature:_____

Name: M.Haris Ikram

Signature:_____

Name: Noor uz Zaman

HITEC University, Taxila

Session 2022-2026

Dedication

This thesis is wholeheartedly dedicated to our respected supervisor, **Dr. Sobia Arshad**, whose constant support, expert guidance, and insightful feedback made this project a reality.

We extend our sincerest gratitude to you, Dr. Sobia Arshad, for your commitment and hard work throughout our journey. Your in-depth knowledge of microprocessors and interfacing techniques provided a strong foundation for our research. From the development of the initial idea to the final deployment, your input played a crucial role in shaping both the direction and the depth of our work.

Your encouragement, constructive criticism, and patience motivated us through the most challenging phases. You continuously inspired us to think critically, stay focused, and uphold high standards in our research. Your confidence in what we can accomplish together empowered us to go the extra mile and reach goals that truly matter.

This achievement in my career stands as evidence to the impact of your mentor ship. Thank you for shaping our lives both academically and personally; it has truly meant the world to us, and we are thankful we were able to learn from you.

Acknowledgement

To begin with, our most credit is offered to Dr. Sobia Arshad, our respectable supervisor, because of her guidance, her technical knowledge, and firm support were the backbone of our success in completing the Final Year Project on IoT Based Smart Robotic Assistance for Alzheimer's Patients. The project was deeply shaped by her vision and dedicated commitment to excellence. Her mentoring throughout each and every aspect of the project, starting from planning and concluding with implementation, enabled us to deal with project challenges in a directed manner.

The support and motivation we received from her have made this project feasible. We consider ourselves quite lucky to gain the opportunity to work under her guidance and we will appreciate the lessons from this journey for the rest of our lives.

Abstract

The Alzheimer's patients often suffer from free and frequent falls, wandering at night and memory lose because of these issues they may fail to recognize their close family members and forgot to take their medicines regularly which is very dangerous for their health . Existing solutions are limited to either wearable or voice assisted devices but not combined of both plus mobility. The proposed project is capable for continuous monitoring, autonomous patient following, real time fall detection, voice interaction with patient, cloud interaction and medication reminders of patient to his/her caregiver. The integration of robotic knowledge, sensors and IoT communication make this possible. For robust computing vision we utilized **Raspberry pi 5** plus CSI camera along with cloud based alerts. This project ensures the maximum independence for Alzheimer's patient.

Table of Contents

Chapter 1 15
INTRODUCTION 15
1.1 Problem Statement: 15
1.2 Objectives: 17
1.3 Features of Proposed System: 18
1.4 Summary: 18
Chapter 2 20
LITERATURE SURVEY 20
2.1 Gaps Identified in Current Systems: 21
2.2 BiMP Dataset (Multimodal Fall Detection) Paper 1: 22
2.3 YOLOv11-Based Fall Detection with Voice Assistant Paper 2: 23
2.4 Comparison Table: 23
2.5 Summary: 24
Chapter 3 25
METHODOLOGY 25
3.1 System Overview 25
3.2 Project Architecture 25
3.3 Flow Diagram for Patient Tracking/Following: 26
3.4 Implementation flow for fall detection: 27
3.4.1 Human Fall Dataset Creation: 28
3.4.2 Activity Diagram for Fall Detection using basic python conditions: 30
3.4.3 Flowchart for Fall Detection using basic python conditions: 34
3.4.4 Our Approach vs Base Paper Approach of fall detection using basic python: 40
3.5 Flowchart for family member recognition module: 43
3.6 Algorithm for Environment Monitoring: 44
3.7 Use case Diagram 45
Chapter 4 46
IMPLEMENTATION AND RESULTS 46
4.1 Hardware Components: 46
4.1.1 Raspberry pi 5/ 4GB: 46
4.1.2 Raspberry pi 5 casing 49
4.1.3 Raspberry pi5 Power Adapter: 50
4.1.4 SD Card 128 GB 50
4.1.5 Raspberry pi 5 Camera Modules 51
4.1.6 Esp32 Microcontroller 51
4.1.7 Motor driver modules 52

4.1.8 DC Gear Motor 52
4.1.9 Robotic Wheels 53
4.1.10 Bluetooth speakers 53
4.1.11 USB microphone 54
4.1.12 12v external battery pack 54
4.1.13 MQ-2 Sensor 55
4.1.14 MQ-5 Sensor 55
4.1.15 LM35 Sensor 56
4.1.16 Ultrasonic Sensor 56
4.1.17 Servo Motor 57
4.1.18 Vero-Board and soldering Material 57
4.1.19 Bread Board and Connecting Wires 58
4.2 Explanation of Components Grouped by Features 58
4.2.1 Fall Detection 58
4.2.2 Patient Tracking 59
4.2.3 Family Members Recognition 60
4.2.4 Temperature Sensing 60
4.2.5 Gas Sensing 61
4.3 Simulations and Results 61
4.3.1 Gas and Smoke Sensing 61
4.3.2 Temperature Sensing 62
4.3.3 Distance Measuring 62
4.4 Breadboard Implementation and Results: 63
4.4.1 Smoke Detection 63
4.4.2 Gas Sensing 63
4.4.3 Distance measuring and Obstacle Detection 64
4.5 Software models/libraries: 64
4.5.1 Raspberry pi OS installation and Result 64
4.5.2 Python Language (Primary use) 68
4.5.3 Yolon8n-pose Model 68
4.5.4 Yolov8n Object Detection model 69
4.5.5 Opencv 69
4.5.6 Firebase Realtime Database 69
4.5.7 Vosk Speech Recognition 70
4.5.8 eSpeak Text-to-Speech Engine 70
4.5.9 Picamera2 Library 70
4.5.10 Numpy 70

4.5.11 GPIO Library 70
4.6 Modules implementation 71
4.6.1 Patient Tracking Module implementation 71
4.6.2 Patient fall detection module implementation 73
4.6.3 Family member recognition module implementation 75
4.6.4 Medication Alert module implementation: 76
4.6.5 App Development, Integration with Hardware via Firebase module implementation: 76
4.6.6 Firebase Realtime Database Integration: 79
4.6.7 Data Flow to Android App: 80
4.6.8 Android App Architecture & UI: 81
4.6.9 Implementation Checklist: 82
4.6.10 Live Streaming Module Implementation: 83
4.7 Results: 84
4.7.1 Fall detection results 84
4.7.2 Family Member Recognition Results: 85
4.7.3 Results for scheduled medication alerts 86
4.7.4 Environmental monitoring module results 87
4.7.5 Live streaming module results 88
4.7.6 App integration results 89
Chapter 5 90
ENVIORNMENT AND SUSTAINABILITY 90
Chapter 6 91
CONCLUSION 91
6.1 Complexities and Lessons Learned: 91
6.2 Future Work: 91
REFERENCES 92

List of Figures

Figure 1. Price of currently systems present in market. 17
Figure 2. Architecture diagram of entire project. 26
Figure 3: Flow diagram for patient tracking and following. 27
Figure 4: Implementation flow of fall detection. 28
Figure 5: Activity diagram for fall detection using basic python conditions 33
Figure 6: Flowchart for fall detection. 39
Figure 7: Risk Assessment. 39
Figure 8: Flowchart for fall detection. 39
Figure 9: Formulas and calculation thresholds used for predicting the risk factor of patient fall. 40
Figure 10: Flowchart for family member's recognition 43
Figure 11: Algorithm for hardware component 44
Figure 12: Use case diagram. 45
Figure 13: Raspberry pi 5 (4 GB) unboxing 47
Figure 14: RP1 I/O Controller of raspberry pi 5, 4 GB 47
Figure 15: Broadcom BCM2712 quad-core Arm Cortex A76 processor @ 2.4GHz 47
Figure 16: Dual 4Kp60 display ports 48
Figure 17: UART debug port 48
Figure 18: C-type port for powering to pi5 48
Figure 19: USB and Ethernet Port for pi 5 49
Figure 20: Camera connection ports of pi5 49
Figure 21: GPIO Pins of pi5 49
Figure 22: White-pink casing of raspberry pi 5 (4 GB) 50
Figure 23: Power adapter of raspberry pi 5 50
Figure 24: SD Card slot for pi 5 /4gb 51
Figure 25: Pi camera v3.1 with ribbon 51
Figure 26: Esp-32 microcontroller of environmental monitoring 52
Figure 27: BTS7960 Motor Driver 52
Figure 28: DC Gear Motor 52
Figure 29: Blue rubber wheels 53
Figure 30: Bluetooth speakers 53
Figure 31: Microphone for patient response 54
Figure 32: 12v/7A external rechargeable battery pack 54
Figure 33: MQ-2 Sensor 55
Figure 34: MQ-5 Sensor 55
Figure 35: LM35 Temperature Sensor 56
Figure 36: Ultrasonic Sensor 56
Figure 37: Servo Motor 57
Figure 38: Soldering Material and Vero-Board 57
Figure 39: Breadboard and Connecting Wires 58

Figure 40: Grouped components for fall detection 58
Figure 41: Grouped components for patient tracking 59
Figure 42: Grouped component for family member's recognition module 60
Figure 43: Grouped components for temperature sensing. 60
Figure 44: Grouped components for gas sensing 61
Figure 45: Implementation and result of smoke and gas sensor on simulator 61
Figure 46: Implementation and result of temperature sensor on simulator 62
Figure 47: Implementation and result of ultrasonic sensor on simulator 62
Figure 48: Implementation and result of smoke sensor on breadboard 63
Figure 49: Implementation and result of gas sensor on breadboard 63
Figure 50: Implementation and result of ultrasonic sensor on breadboard with servo motor. 64
Figure 51: Raspberry pi 5 imager view 66
Figure 52: OS version selection 66
Figure 53: Hostname of pi 5. 66
Figure 54: Select location and time zone of pi5. 66
Figure 55: Internet setup of pi5. 67
Figure 56: SSH enabling of pi 5 67
Figure 57: Raspberry pi 5/4GB OS installation and desktop interface after bootup. 68
Figure 58: 17 key points detection using yolov8n-pose model 68
Figure 59: Yolov8n human detection and bounding box 69
Figure 60: Architecture of patient tracking module. 71
Figure 61: Safety functions block of patient tracking 73
Figure 62: Architecture of fall detection 73
Figure 63: Architecture of family member's recognition module 75
Figure 64: fall detection results 84
Figure 65: Family member recognition results 85
Figure 66: Scheduled Medication Alerts Module Results 86
Figure 67: Environmental Monitoring Module Results 87
Figure 68: Live streaming results 88
Figure 69: App results 89

List of Tables

Table 1:- Comparison between various existence systems 21
Table 2: Showing the phase 01 of dataset creation. 29
Table 3: Showing the phase 02 of dataset creation. 29
Table 4: Showing the total videos of both phases and final dataset total videos. 29
Table 5: Showing the data in each class of final dataset. 30
Table 6: Showing the comparison for keypoints mapping values. 40
Table 7: Showing the comparison for CoG calculation. 41
Table 8: Showing the comparison for body angle calculation. 41
Table 9: Showing the comparison for hip-shoulder relationship. 41
Table 10: Showing the H/W ratio factor calculation 41
Table 11: Showing the threshold comparison. 42
Table 12: Showing the CoG velocity calculation 42
Table 13: Showing the overall algorithm comparison. 42
Table 14: Showing the screen distribution for tracking the human motion 72
Table 15: Threshold values for forward/stop/backward motion 72

Chapter 1

INTRODUCTION

Alzheimer's disease is a progressive neurological disorder that gradually damages brain function over time. The disease causes the degeneration of brain cells and disrupts the communication pathways between neurons. In addition, the accumulation of abnormal proteins, such as amyloid plaques and tau tangles, contributes to cognitive decline. As the condition advances, patients experience memory impairment, difficulty in decision-making, behavioral changes, and a reduced ability to perform everyday activities independently [1]. Consequently, many individuals become increasingly dependent on family members and caregivers for daily support.

Patients suffering from Alzheimer's disease frequently forget to take prescribed medications, may leave electrical appliances switched on, or unintentionally wander away from their homes. These situations can expose patients to significant safety risks and create additional stress for caregivers. Since continuous human supervision is not always feasible, there is a growing need for an intelligent assistance system capable of monitoring patients, providing reminders, and communicating important information to caregivers with minimal human intervention.

This project proposes an Internet of Things (IoT)-based smart robotic assistant designed specifically for individuals affected by Alzheimer's disease. The proposed system is intended to monitor patient activities, generate medication and task reminders, and notify caregivers in emergency situations. By integrating robotics, sensing technologies, and intelligent communication mechanisms, the system aims to provide a reliable, responsive, and efficient healthcare assistance platform. The ultimate objective is to improve patient safety, reduce caregiver workload, and enhance the overall quality of life of Alzheimer's patients.

1.1 Problem Statement:

Individuals with Alzheimer's disease face numerous challenges, including memory loss, medication noncompliance, wandering behavior, and difficulties in maintaining a safe living environment. These challenges often increase their dependence on caregivers and may result in

accidents, injuries, or other hazardous situations. Among these concerns, falls and wandering incidents are particularly common and can have serious consequences.

Caregivers require an effective method of receiving real-time information regarding a patient's condition and activities. Therefore, a mobile application connected to the robotic system is needed to provide continuous updates, alerts, and notifications. Such a system would allow caregivers to remotely monitor patients and respond promptly when assistance is required.

The proposed robotic assistant is expected to support live video monitoring and accurately follow the designated patient within a predefined operational range. Maintaining a stable communication link between the robot, cloud services, and caregiver application is essential for ensuring reliable system performance and timely delivery of critical information.

Existing solutions suffer from several limitations:

- **Wearable Devices**: Many patients find **wearable devices uncomfortable, may forget to wear them, or refuse to use them altogether**. Additionally, these devices primarily focus on location tracking or fall detection and generally lack interactive reminder and environmental monitoring capabilities. Example include Freestyle Libre, Philip Wearable Biosensors, VivaLNK Wearable Monitor and Apple Watch fall-detection system.
- **Voice Assistants**: Mostly Conventional voice assistants such as Amazon Alexa and Google Home are **fixed in one location and can't accompany patient throughout different areas** of a home. Furthermore, they typically lack advanced health-monitoring features.
- **Companion Robots:** Several commercial available robots are designed mainly for companionship and entertainment purposes. While they **may provide social interaction, they often do not offer comprehensive heath monitoring or fall-detection functions.** Example include Buddy Robot and ElliQ Robot.
- **GPS Tracker and Smart Wearable:** These solutions mainly p**rovide location information and offer limited interaction with patient.** Their functionality is generally restricted to tracking and basic notifications. Example include AngelSense and GPS SmartSole.
- **Highly Cost/ Expensive:** Many healthcare robots and monitoring systems available in market

are expensive, making them inaccessible for a large number of patient and families. Like shown in figure 01 below in PKR 1,035440.

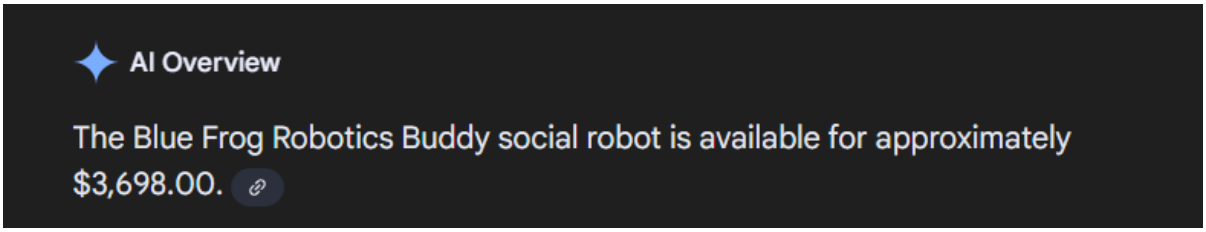


Figure 1. Price of currently systems present in market.

1.2 Objectives:

The primary objective of this project is to develop a secure, feature-rich robotic assistant based on Raspberry Pi 5 technology that supports patient independence and intelligent environmental monitoring. The specific objectives are listed below:

1. Design and Develop a cost effective IoT-based robotic assistance for Alzheimer's Patient by integrating robotics, sensors and intelligent communication technologies.
2. Incorporate a reliable fall detection mechanism in to system.
3. Provide voice-based interaction and medication reminders functionality.
4. Enable real time environmental monitoring using IoT sensors.
5. Develop a mobile application for caregivers to facilitate remote patient monitoring.
6. Ensure seamless communication among hardware component, software platforms and cloud services while maintaining data security.
7. Implement the obstacle-avoidance functionality and maintain a safe distance between the robot and patient.
8. Achieve satisfactory battery life for extended operation.
9. Develop a reliable, user-friendly and economically viable solution.

These objectives ensure that the proposed system remains both technically effective and easy to use for caregivers and non-technical users.

1.3 Features of Proposed System:

The key features include:

1. **Real-time fall detection**

   The system utilizes the **YOLOv8-Nano** pre-trained model for human keypoint detection. Computer vision processing is performed using the **raspberry pi 5** microcontroller, while a CSI camera is employed for continuous patient monitoring and image acquisition.

2. **Patient Tracking and Following**

   The robot incorporates wireless communication capabilities and mobility functions to track and follow the designated patient autonomously within the operating environment.

3. **Immediate caregiver Alerts on Android App**

   Microphones and speakers are used to facilitate voice interaction. Upon detecting a fall or emergency event, the system generates notifications and alerts that are transmitted to the caregiver application through cloud services.

4. **Obstacle-Aware Navigation**

   Ultrasonic sensors are integrated into the robot to detect surrounding objects, maintain a safe distance from obstacles, and support smooth navigation while following the patient.

5. **Battery Powered Mobility**

   A suitable lithium-ion battery pack is utilized to provide sufficient operating time and ensure uninterrupted system functionality.

6. **Environmental Monitoring**

   The robot continuously monitors environmental conditions using sensors such as MQ-2 for gas detection, MQ-5 for smoke detection, and LM35 for temperature measurement.

1.4 Summary:

This chapter introduced the proposed IoT-based robotic assistant for Alzheimer's patients and highlighted the challenges associated with memory loss, wandering behavior, falls, and medication management. The proposed system combines robotics, intelligent sensing, communication technologies, and remote monitoring capabilities to address these challenges. By

enhancing patient safety, reducing caregiver burden, and promoting independent living, the system aims to provide a practical, cost-effective, reliable, and user-friendly solution for Alzheimer's care.

Chapter 2

LITERATURE SURVEY

In order to provide continuous assistance to Alzheimer's patients who experience memory impairment, wandering behavior, falls, and medication-related issues, various engineering solutions have been explored in the medical and assistive technology domain. Previous efforts include the development of several wearable and monitoring devices such as AngelSense, SafeWander, GPS SmartSole, and MindMe Locate. However, due to the cognitive decline associated with Alzheimer's disease, patients may forget or refuse to wear such devices, which significantly increases the risk of accidents, injuries, and unsafe situations.

To address these limitations, this project focuses on identifying suitable IoT-based sensors and communication technologies to develop an intelligent robotic system capable of tracking patients, generating real-time alerts, and notifying caregivers during emergency conditions.

AngelSense is a commercial assistive technology device designed for individuals with special needs, particularly those prone to wandering. It functions as a GPS-based tracking system combined with a speakerphone, enabling continuous location monitoring. The system operates throughout the day regardless of app activity and provides detailed location history, maps, and event logs, allowing caregivers to review movement patterns. It is widely used for maintaining safety and connectivity with loved ones [4].

SafeWander is a wearable safety device designed to detect motion changes such as a patient attempting to get out of bed or a chair. The device is securely attached to clothing and sends immediate alerts to caregivers through a connected mobile application when movement is detected, helping to prevent fall-related incidents [5].

GPS SmartSole is a discreet GPS tracking solution integrated into footwear. It is specifically designed for individuals with Alzheimer's disease who may wander unintentionally. The system allows caregivers to define safe zones, and when the wearer exits these predefined boundaries, alerts are sent via SMS or email notifications [6].

Amazon Alexa is a voice-based intelligent assistant capable of supporting interaction in home environments. It can respond to voice commands and assist users in daily activities. Its functionalities include controlling smart home devices, playing music, setting reminders and alarms, managing schedules, and providing general information such as news and weather updates. It is often used as a companion tool for elderly individuals who require conversational engagement and basic assistance [7].

Buddy Robot is an emotionally expressive companion robot developed by Blue Frog Robotics. It is designed to interact naturally with users by displaying a range of emotional behaviors, from happiness to frustration. In addition to companionship, it supports communication, entertainment, education, and video calling features, making it a multifunctional assistive device [8].

ElliQ is a senior-oriented companion robot designed to support elderly users in maintaining their daily routines. It provides medication reminders, exercise suggestions, and wellness guidance to promote long-term physical and mental health. Additionally, it enables communication through messages, photo sharing, and social engagement activities such as games and virtual community events, helping users stay connected with family and caregivers [9].

Although wearable devices and voice assistants provide partial solutions for monitoring and assistance, they generally lack mobility and continuous real-time physical tracking of the patient. Wearables are often limited to detection or location tracking, while voice assistants remain stationary and cannot follow the patient across different environments. This creates a gap in existing systems, highlighting the need for a more integrated and mobile solution.

2.1 Gaps Identified in Current Systems:

A comprehensive review of existing healthcare assistance systems indicates that several challenges remain unresolved. While many commercial solutions demonstrate effective performance in individual applications, they are often financially inaccessible to many users. In addition, the majority of these systems lack seamless integration of autonomous mobility, real-time patient tracking, environmental monitoring, and automated emergency notification features. The following table 01 is a comparison of these systems with the design suggested.

Table 1:- Comparison between various existence systems

System/Product	Overview	Technology Used	Key Features	Limitations	Proposed system improvements
GPS tracker bands		GPS, Bluetooth	Track patient's location, send alerts.	No reminders and interaction.	Adds interactive robot + reminders + IoT + alerts
Amazon Alexa/Google Home		Voice Assistant, Cloud IoT	Voice-based reminders, home automation	Stationary device, cannot follow or monitor patient	Movement + tracking
Buddy Robot		Camera , Wi-Fi	motional Support, Reminders	No IoT health monitoring, or patient following.	Integrates IoT sensors + tracking + caregiver alerts.
ElliQ Companion Robot		AI cloud	Engage elderly via conversation	No real time location	Mobility feature
Proposed Smart Robotic Assistant		IoT, Sensors, AI, Robotics	Patient Tracking, Monitoring, reminders, IoT connectivity		Combined mobility (on wheel), IoT, and intelligent patient assistance in one system

2.2 BiMP Dataset (Multimodal Fall Detection) Paper 1:

The BiMP dataset was developed to support research in fall detection by providing a multimodal collection of synchronized audio and video recordings. The dataset contains both fall incidents and routine daily activities captured in different environments, enabling researchers to evaluate the effectiveness of detection algorithms under diverse conditions. To protect participant privacy, measures such as facial anonymization were incorporated during data preparation.

To validate the dataset, the researchers employed several deep learning approaches, including GoogLeNet for feature extraction and Long Short-Term Memory (LSTM) networks for temporal analysis. Audio information was processed using signal analysis techniques such as the Short-Time Fourier Transform (STFT) and Continuous Wavelet Transform (CWT). System performance was assessed using commonly accepted evaluation metrics, including accuracy, precision, and recall. Experimental findings indicated that combining audio and visual information produced

better detection results than relying on a single modality. Consequently, the dataset provides a valuable resource for the development of fall detection solutions in areas such as elderly care, smart environments, and robotic monitoring systems.

2.3 YOLOv11-Based Fall Detection with Voice Assistant Paper 2:

This study proposes a real-time fall detection framework that combines the YOLOv11 pose estimation model with a voice-assisted interaction system. Human body keypoints are extracted from video streams to analyze posture-related characteristics, including body inclination and center-of-gravity movement, which are used to determine whether a fall has occurred.

Once a fall event is identified, the system initiates a voice-based interaction to verify the user's condition. The user is prompted to respond, and if no response is detected within a specified duration, an emergency notification is automatically transmitted to caregivers. This verification process minimizes false detections and enhances the overall dependability of the system. Experimental results indicate that YOLOv11 achieves superior detection accuracy and faster processing performance compared to earlier versions such as YOLOv7. These characteristics make the proposed approach suitable for healthcare monitoring, elderly assistance, and intelligent surveillance applications.

2.4 Comparison Table:

Feature	**Paper 1 (BiMP Dataset)**	**Paper 2 (YOLOv11 System)**
Main Focus	Dataset creation	Real-time system
Approach	Multimodal (Audio + Video)	Vision + Voice interaction
Models Used	GoogLeNet, LSTM, STFT, CWT	YOLOv11
Data Type	Pre-collected dataset	Live video input
Detection Method	Deep learning classification	Pose estimation + rules
Special Feature	Audio-visual fusion	Voice confirmation

Accuracy	Validated dataset performance	Higher real-time accuracy
Real-time Capability	Limited	Yes
False Alarm Handling	Basic	Improved with voice
Use Case	Training ML models	Practical deployment

2.5 Summary:

This chapter discusses that existing assistive technologies such as wearable, voice assistants and companion robots offers the limited features, focusing on location tracking, basic reminders or emotional companionship.

Chapter 3

METHODOLOGY

This chapter describes the arrangement of the system in such a manner that the project goals would be achieved. The system is designed to make human life independent, easy and secure.

3.1 System Overview

When we think of an embedded system, generally, we will consider how the input data can be of any kind; such as images, text, and video. And can be sent and received from any type of device/sensor. After processing that data, we will generate an output in either the form of images, videos or text.

3.2 Project Architecture

In this section, we highlight the different components that are part of our system, and how they connect and transfer data to each other. Our inputs will be divided by their hardware compatibility and since **ESP32** is not going to be very good at handling much vision-based input data — the processing of vision-based input data such as video data being captured and sent to our model for decision making will occur directly on the **Raspberry pi 5 board**, since that has the capability of working with real-time video. On the other hand, the main points of not using the ESP32 will be that all of the non-vision-based input data will be lightweight enough that it could be handled by the ESP32, therefore allowing us to keep the hardware much smaller and less costly compared to using any other hardware for these types of inputs; therefore, we will select our hardware based upon the complexity of the input data.

We will input data into Raspberry Pi 5 and ESP32, which will process those inputs using the predetermined **Yolov8n pose recognition algorithm**; these algorithms consist of identifying 17 keypoints on a human body. Then we will analyze those keypoints using an algorithm designed to detect a fall and help the embedded sensors from each device send both the identified keypoint data and fall detection status to the Rest API. For backups, data will first be sent to the Rest API as a cloud-based resource; if for whatever reason a user accesses the Application (i.e. Care Giver) without a connection to the Rest API, the Care Giver will not receive any data about falls unless it

was received directly from the embedded devices connected to an Internet-connected device. We will also create a **two-way verbal communication link between the Robot and the Patient** so each party (robot and patient) can confirm whether a fall occurred or not; by identifying a fall occurrence through verbal communication, we can recognize any fall alerts sent by the embedded sensors are true. We will require providing caregivers with access to review sensor data aside from using the two-way communication method with patients.

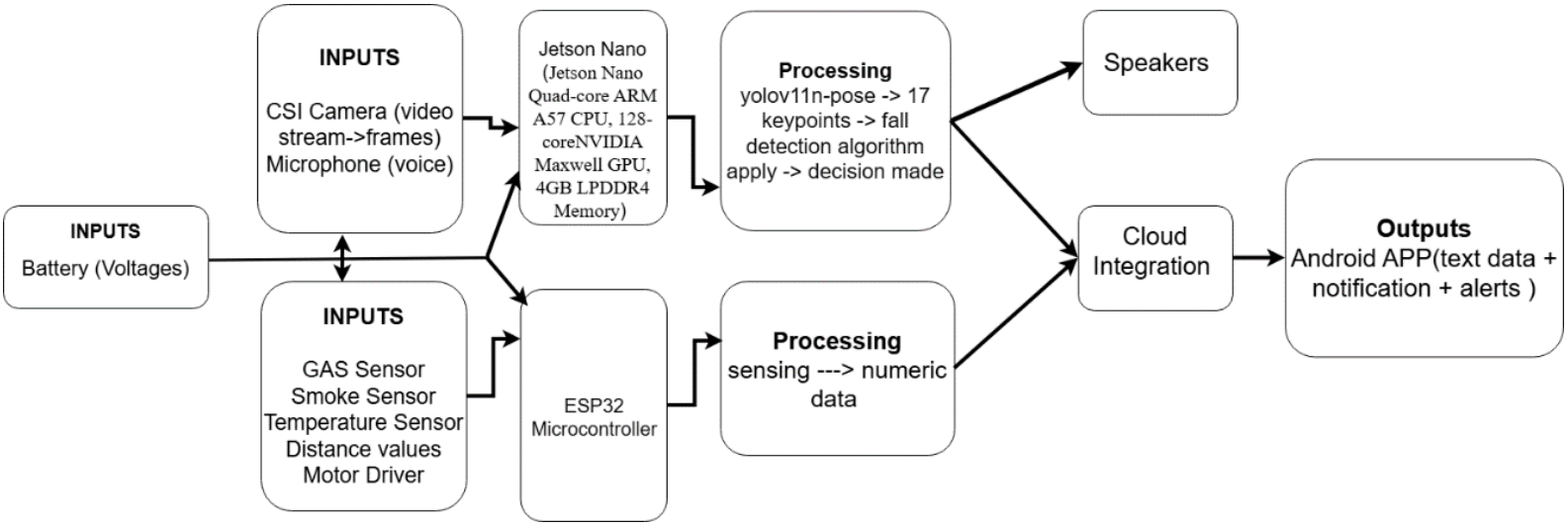


Figure 2. Architecture diagram of entire project.

3.3 Flow Diagram for Patient Tracking/Following:

This section briefly describes how the main module of our project will track patients with a robotic system. The mobility of the robot is very important because it will continually monitor a patient at home while the patient is alone (without a caregiver or family member). The flow of this part of the implementation is shown as Figure 3. The first thing that we do is acquire the image, meaning that we will capture an input frame from video camera streaming. The deep learning model will identify the human in the frame, and determine where they are located by mapping them in the coordinates of the image: this will allow the robot to determine its next actions to follow the patient, such as moving left, moving right, moving forward, moving backward, and/or stopping at the patient's location. There are safety measures implemented into this module: if the patient falls, the robot will stop immediately; if the patient goes missing from the image, the robot will stop and

call for the patient through its speakers; and if the robot becomes confused about which person to track, it will ask the patient for their location.

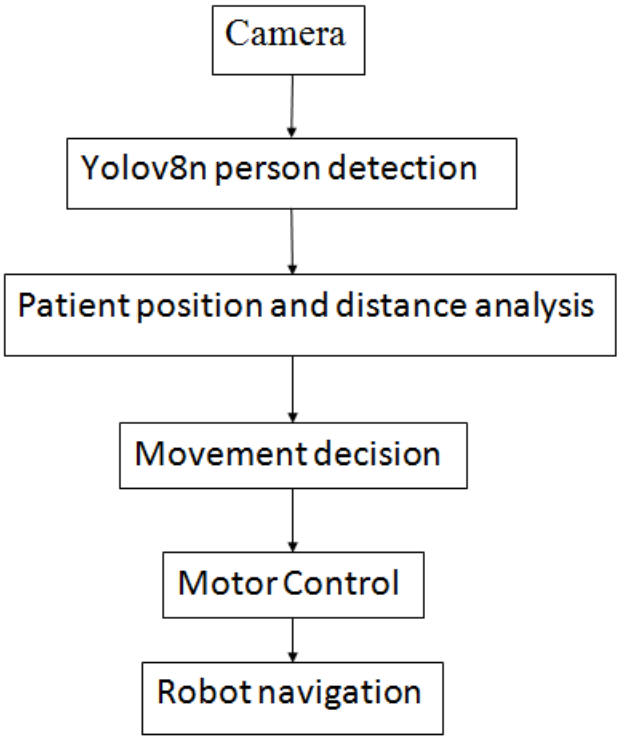


Figure 3: Flow diagram for patient tracking and following.

3.4 Implementation flow for fall detection:

In this module, we will first capture patient through camera input, then we will apply another model from YOLO family of **yolov8n-pose** for detecting human as well as keypoints of human. With the help of keypoints we will analyze human current posture and then we will find **center of gravity and velocity** of human motion to predict suspicious motion. Then based on **4-state fall detection model** we will predict fall, then to avoid false positive results we will perform voice interaction between patient and robot, after this confirmation robot will decide either to send alert or not, to caregiver.

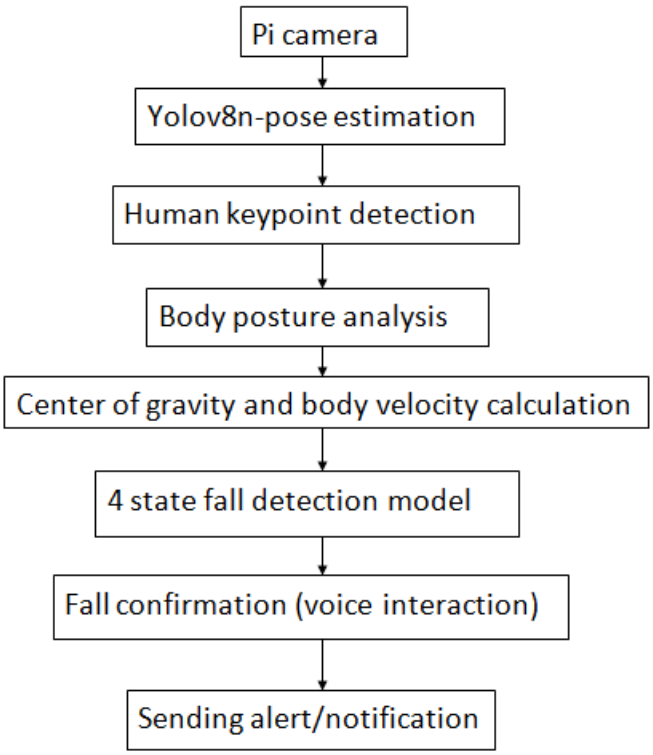


Figure 4: Implementation flow of fall detection.

3.4.1 Human Fall Dataset Creation:

This module consists of taking an image from a patient and inputting it into a camera system: we will then input that patient image into a second model (Yolov8n-Pose) where the system will then use that image data to identify the patient and locate marker points on the patient (these points are defined as anatomical key points) that we will then use to analyze the current position of the patient and from those data points create the patient's center of mass, velocity and from those pieces of information we will identify suspicious movements. Also using a four state fall detection model, we will use that model to predict a fall and in order to reduce the chances of false positives the robot will engage in verbal communication with the patient before the robot makes a decision whether or not to send an alert to a caregiver.

In the following sections we will provide a description of the dataset which is defined as a collection of related and ordered data elements; when we take that ordered data and put it into models we will be able to have the model learn from the data in order to create a meaningful result that will be useful to researchers and engineers.

Phrase 01:

Table 2: Showing the phase 01 of dataset creation.

Classes	Total videos	Place 1 Left angle Member 1	Place 2 Right angle Member 2	Place 3 Front angle Member 3	Place 4 Back angle Member 4
Falling	40	10	10	10	10
Sitting	40	10	10	10	10
Walking	40	10	10	10	10
Sleeping	40	10	10	10	10
Total	**160**				

Now come to phase 02, in this phase we keep our parameters same as we used different locations, angles, persons to make it generalized dataset. Main purpose of this phase is to enhance the data so that we can test fall detection in a very accurate manner. So in phase 02 we got 320 videos more in addition to 160 videos shown in table 3 below. In this phase we keep 80 videos in each class and in **total we got 480 .mp4 videos in our whole customized dataset.** Now after both phases we have data shown in table 4 below.

Phrase 02:

Table 3: Showing the phase 02 of dataset creation.

Classes	Total videos	Place 1 Left angle Member 5	Place 2 Right angle Member 5	Place 3 Front angle Member 6	Place 4 Back angle Member 6
Falling	80	20	20	20	20
Sitting	80	20	20	20	20
Walking	80	20	20	20	20
Sleeping	80	20	20	20	20
Total	**320**				

Table 4: Showing the total videos of both phases and final dataset total videos.

Phase	No of videos

Phase 01	160
Phase 02	320
Total	480

And now for balancing the final dataset, we have 120 videos in each class as we will divide whole dataset into 4 as 480/4 = 120, so each class now has equal data after phase 02 shown in table 5 below.

Table 5: Showing the data in each class of final dataset.

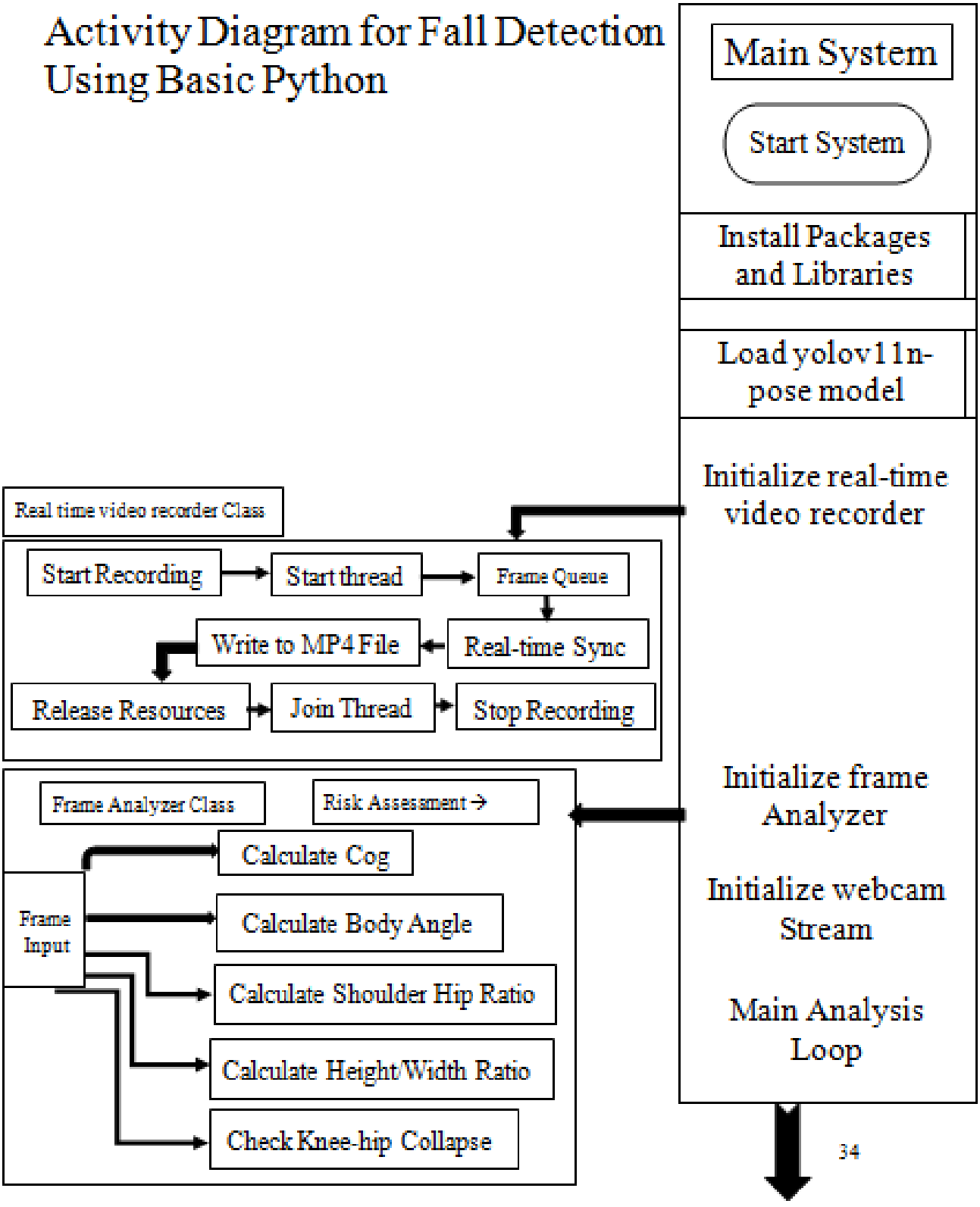
Classes	Total no of videos
Falling	120
Sitting	120
Walking	120
Sleeping	120
Total	480 (same as total in table 12)

3.4.2 Activity Diagram for Fall Detection using basic python conditions:

In this section of the document, the writer will provide a detailed step-by-step explanation of how the code detects when a person has fallen by breaking each section down into logical steps (e.g., install libraries, load model). The first step involves installing and loading the Yolo model. The next step involves capturing an input stream from the camera (video frame) and performing real-time analysis of each successive video frame. This section will also present the five principal variables used for fall detection: Centre of Gravity (COG), Angle Between Shoulder and Hip, The Ratio of the Length Between Shoulder and Hip to the Length Between Knee and Ankle, The Height of the Bounding Box, and The Width of the Bounding Box Additionally, The Velocity of the Falling Object is measured. After calculating these variables, they will be sent to a main analysis loop where the decision about whether or not to send an alert to the caregiver (using voice communication with patient through speakers and/or microphone) will be made.

The ‘main_system’ box serves as a pre-preparation stage before commencing the actual detection of human movements; First, we install critical dependencies such as ultralytics, openCV, and numpy, and then we load the pre-trained YOLO model to estimate the poses of humans, followed by setting up a video recording device that will allow us to display a live feed (real-time recording). Finally, we will define an object to analyze each frame called ‘FrameAnalyzer’ so that we can determine whether each frame has a falling or not falling person in it after we recorded our video.

Activity Diagram for Fall Detection Using Basic Python



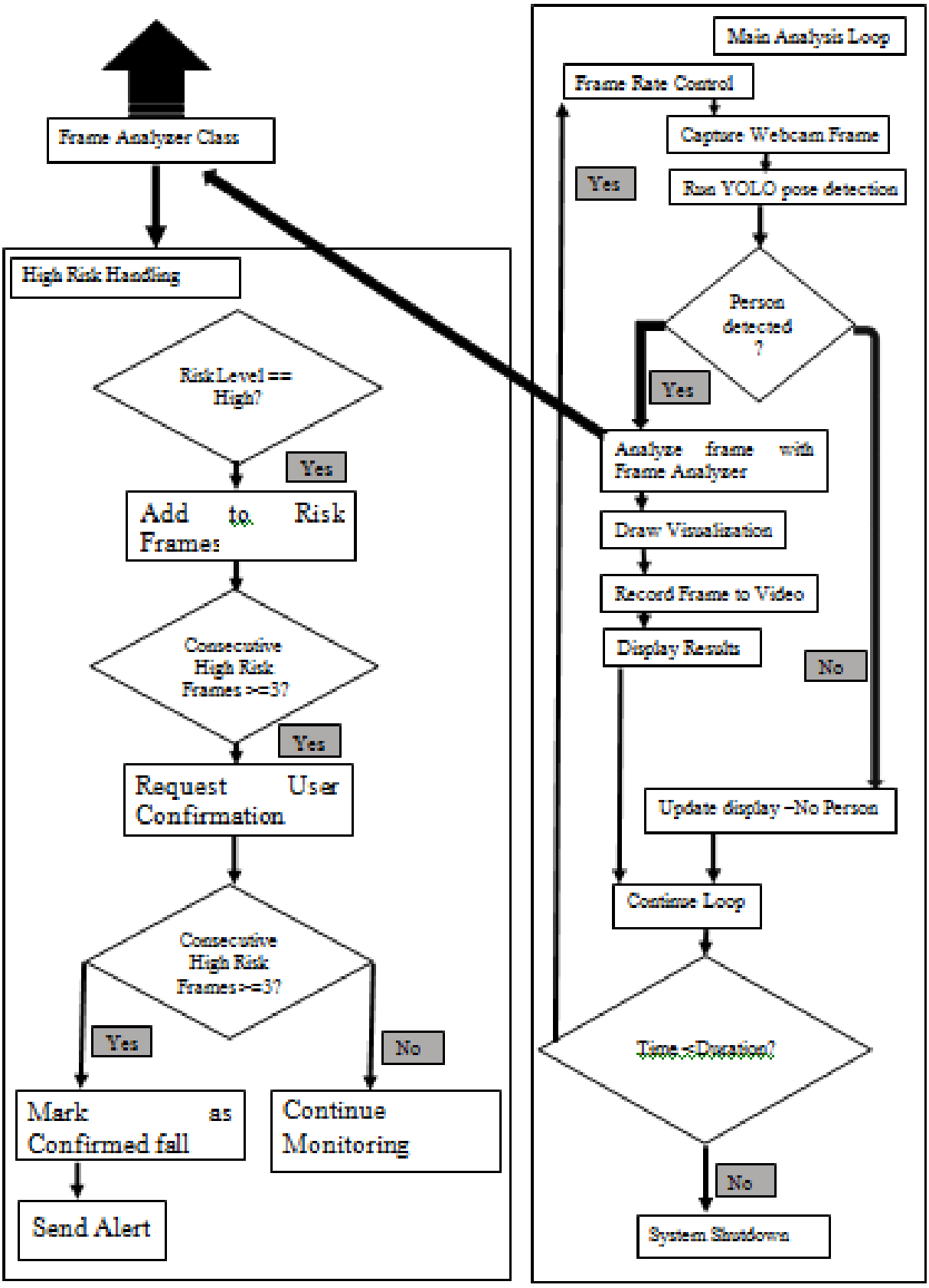

Main Analysis Loop
Frame Rate Control
Capture Webcam Frame
Run YOLO pose detection
Person detected ?
Yes
Yes
Analyze frame with Frame Analyzer
Draw Visualization
Record Frame to Video
Display Results
No
Update display –No Person
Continue Loop
Time.<Duration?
No
System Shutdown
Frame Analyzer Class
High Risk Handling
Risk Level == High?
Yes
Add to Risk Frames
Consecutive High Risk Frames >=3?
Yes
Request User Confirmation
Consecutive High Risk Frames >=3?
Yes
No
Mark as Confirmed fall
Continue Monitoring
Send Alert

Figure 5: Activity diagram for fall detection using basic python conditions

The Main_Analysis_Loop box runs in a continuous loop and will maintain 30 frames per second (FPS) for live video streaming while also classifying whether each frame has a falling or not falling person. The Core processing engine begins executing from this point; a frame from the live webcam feed is used to run a pose estimation model. If the model classifies that there is no person present in the frame, we will not display anything on the screen; however, if the model classifies that a person is present, we will be able to provide the angular measurement of relative motion of the person between two reference points.

To calculate these factors, a class called Frame_Analyzer_Class will be developed in order to change the flow of code into this section of implementation.

When we reach the Frame_Analyzer_Class section, our aim is to input frame data of a person's movement, determine the Center of Gravity (CoG), calculate the angle of the body relative to the ground (generally referred to as the body angle), how the shoulders compared to the position of the hips (hip-shoulder ratio), the measurement between the top and bottom edge of a person (height/width ratio) and how the knee relates to the hip (knee/hip collapse).

Beginning with the calculation of CoG, the CoG velocity will then be calculated.Within this calculation we will first use a time-based analysis for the previous temporal calculations, then we will use this information to establish the risk level associated with the input frame data.Now we will proceed to analyze the High_Risk_Handling section by initially determining whether or not the calculated risk is considered high or low?If the risk is considered high, there will be a check for the last three frames and/or inputted frames are all classified as having a high risk.If the three consecutive frames are determined to be high risk, we will perform a verbal communication check with the patient.

At this time we may ask the patient if they feel healthy or have experienced a fall. If the patient confirms that they have fallen, that frame will be recorded as a "fall detected frame" and the alert will be sent to the caregiver.

As long as the patient hasn't fallen, we will continue to monitor the patient. In the Main_Analysis_Loop block we will perform continuous operations in a loop, we will first maintain

30FPS for Live Streaming, and we will also make sure we are at 30FPS on control. This is where the Core Processing Engine starts. Here we take the image/frame from the webcam, run the pose estimation model on that frame, and here we will make a decision about whether the person is actually in the frame. If they're not, then we will just display that no person has been detected. If they are, we will then calculate all the matrices (e.g. angle, velocity, ratios/differences) for that frame.

This will switch the flow of the code to the Frame_Analyzer_Class (Box 3). At that point, we will select the appropriate frame, calculate the CoG, Body Angle, Hip-to-Shoulder Ratio, Height-to-Width Ratio, Knee-to-Hip Collapse, from the CoG calculate CoG Velocity, perform Temporal Analysis (time-based analysis), and finally from the 1st analysis, calculate the Risk Level for that particular Frame.

The RealTimeVideo Recorder_Cla box have main operation related to video recording. First we will start the recording from webcam, then we will generate the separate thread for handling this task, in frame queue we will store the frame temporarily in queue for maintaining the FIFO operation, then real-time-synchronization will maintain the enter duration rate by the user. Then writing to .mp4 means we will save the frames to video file for observation of results in .mp4.

3.4.3 Flowchart for Fall Detection using basic python conditions:

The flowchart depicted in the figure below illustrates the process for determining if someone has fallen. The system first receives an input from an image frame and uses the confidence score from keypoint detection to determine if the input contains a person (≥ 0.5). Additionally, five different attributes are used to determine an overall risk of fall.

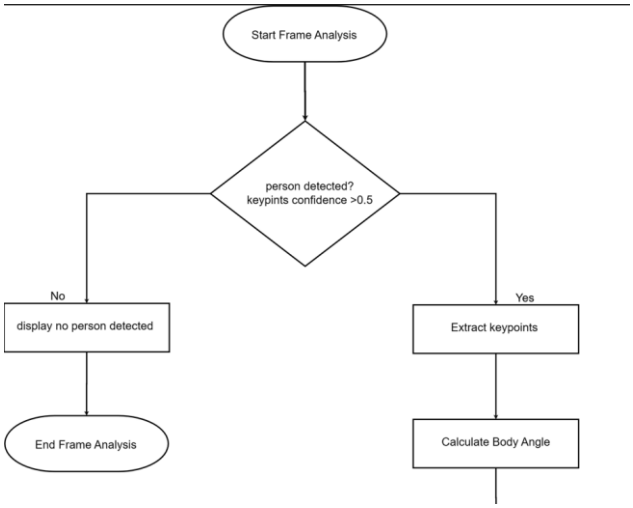
First, the body angle can increase the risk of falling by an additional 25%. If the body angle is <30 degrees, this indicates the individual is not upright (e.g. bent, leaning), whereas if it is ≥30 degrees, this indicates they have an upright posture (e.g. standing straight).

Second, the shoulder/hip ratio can also increase the risk of falling. If the y-coordinate of a person's shoulder is less than their hip, they are at greater risk of falling due to the negative ratio between their shoulder and hip.

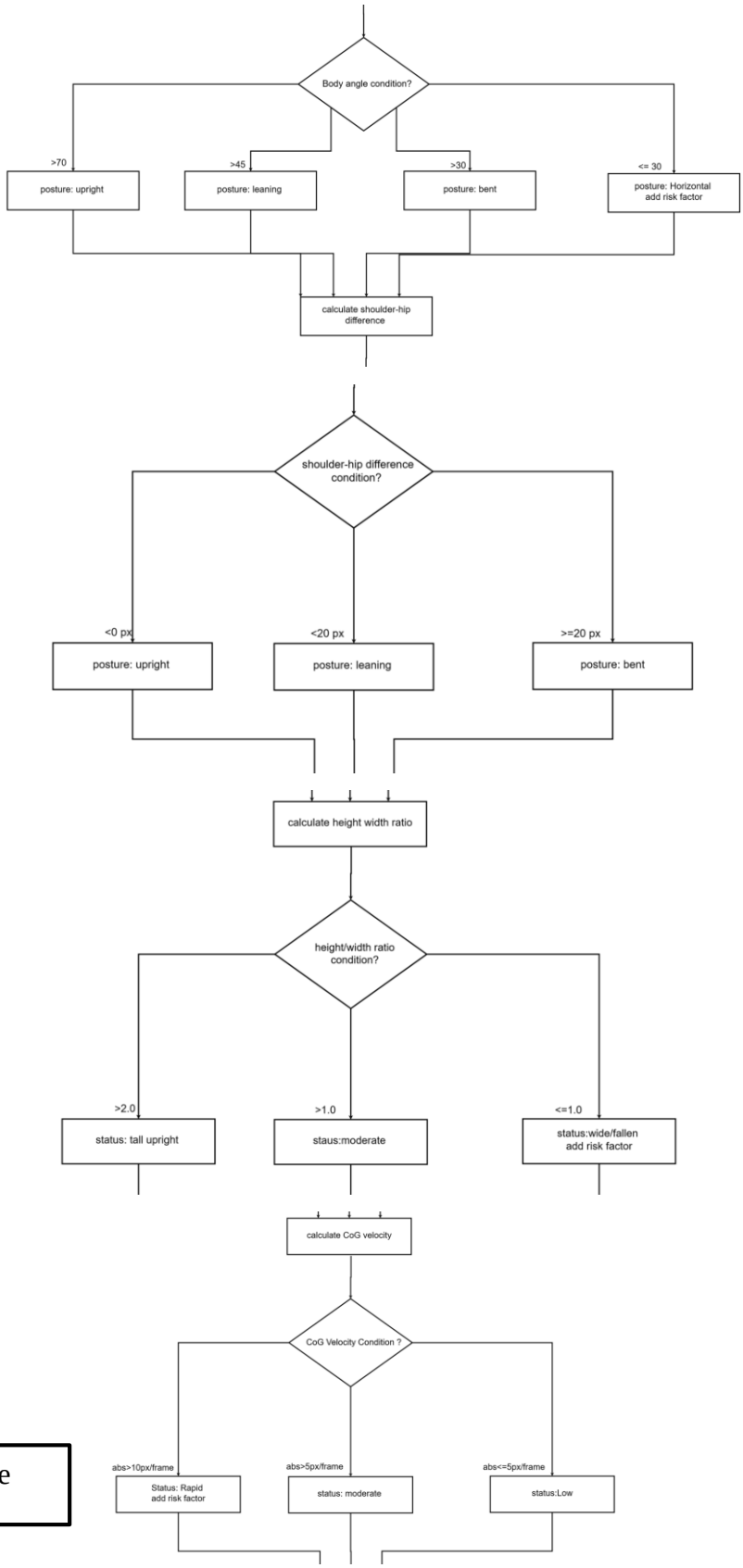
The height/width ratio can give you an idea as to a person's dimensions, i.e., whether or not they are longer than they are wide. The ratio for a standing person who is greater than two times in height than their width could indicate that they are going to be taller than they are wide, whereas when an individual is falling, the ratio of height to width can be less than one; therefore, such an individual would be wider than tall. We can also use the y-coordinates of the hips and knees and the height-to-width ratio to estimate the risk of an individual falling.

In addition, if there is a 25% increase in the risk of falling, due to the centre of gravity (CoG) being too far forward based on the y-coordinates of the hips and knees, then the risk of an individual falling is also increased by 25% due to the CoG velocity (i.e., the CoG velocity is greater than or equal to 10 pixels/frame).

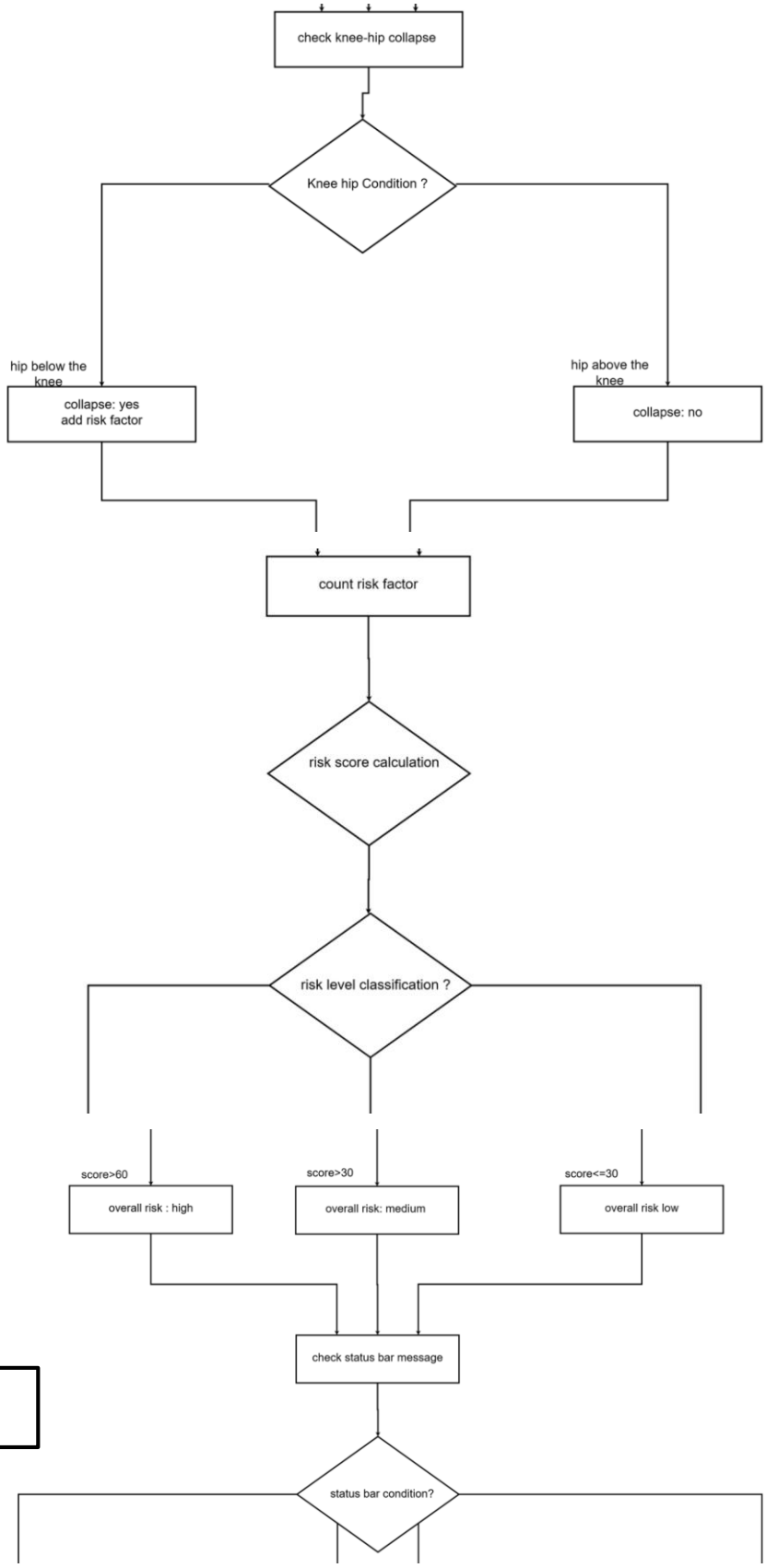
Finally, by using the risk factor described in the above examples, we can calculate the overall risk score and classify the risk level as either low, moderate, or high (as seen below in Figure 6). Each individual true case will increase the risk score by one, and the risk score will be classified based on the aforementioned three risk levels.



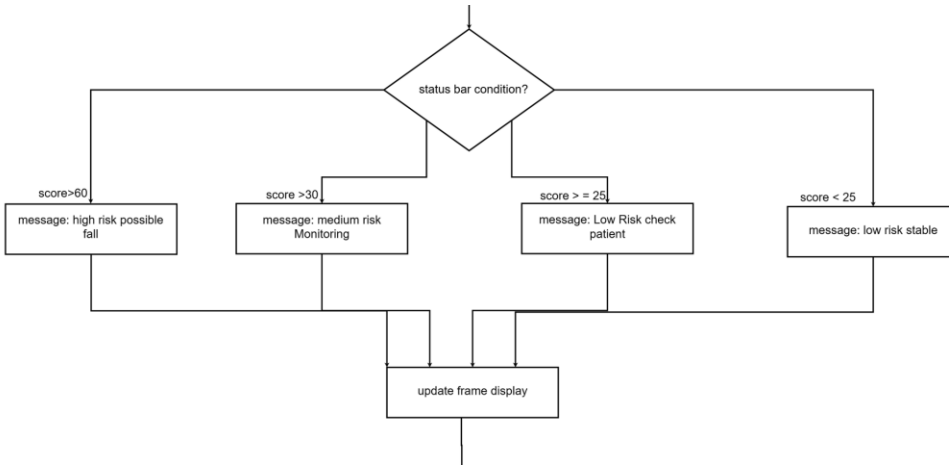
Continue to next page

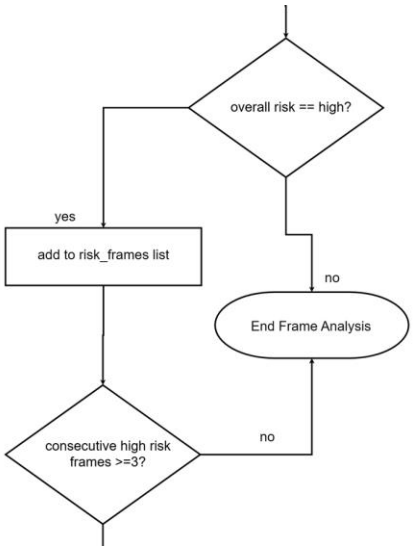


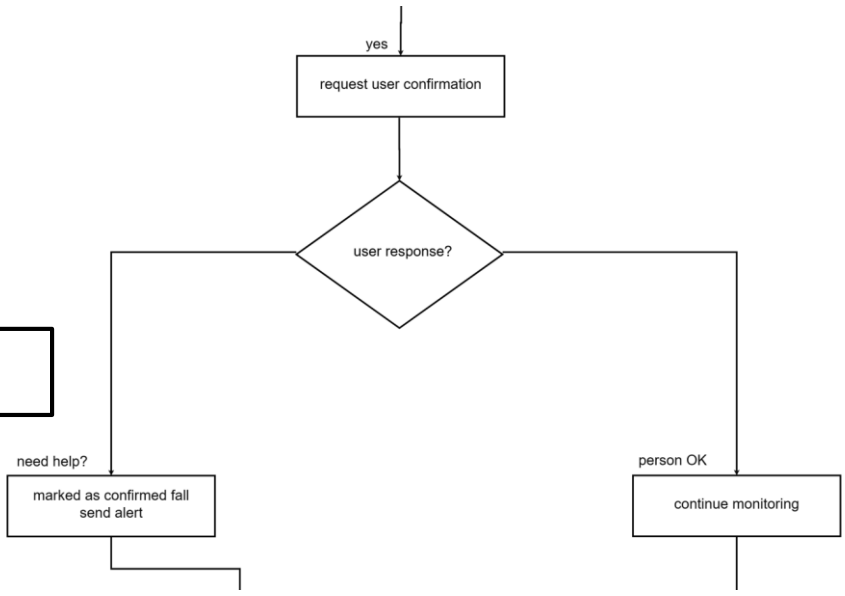
Continue to next page



Continue to next page







Continue to next page

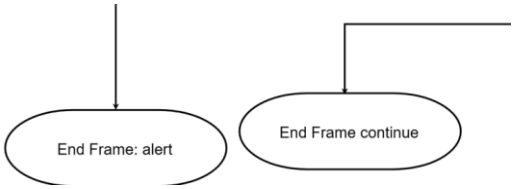


Figure 6: Flowchart for fall detection.

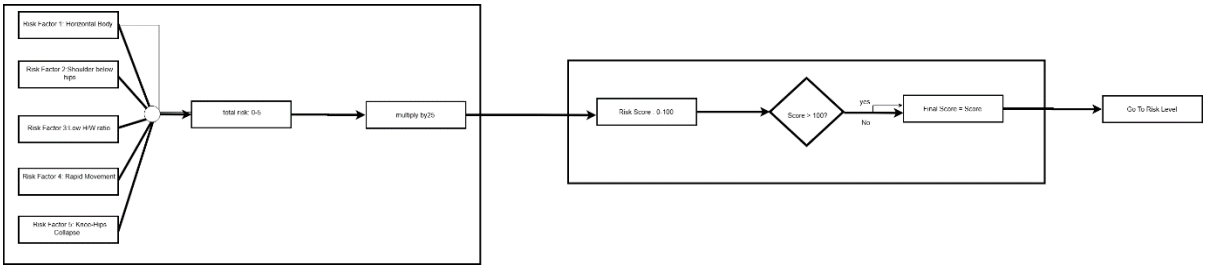


Figure 7: Risk Assessment.

Now if the risk score exceeds from 60 means alarming condition and if less then can be low or medium/moderate case, for 60>score>30, will be predicted as medium risk but score<=25 then stable condition no fall at all shown in image 8 below. Below we have equations/formulas used for calculating these factors. Left/Right shoulders hips and knees will be utilized for making the decision instead of all 17 keypoints shown in figure 9.

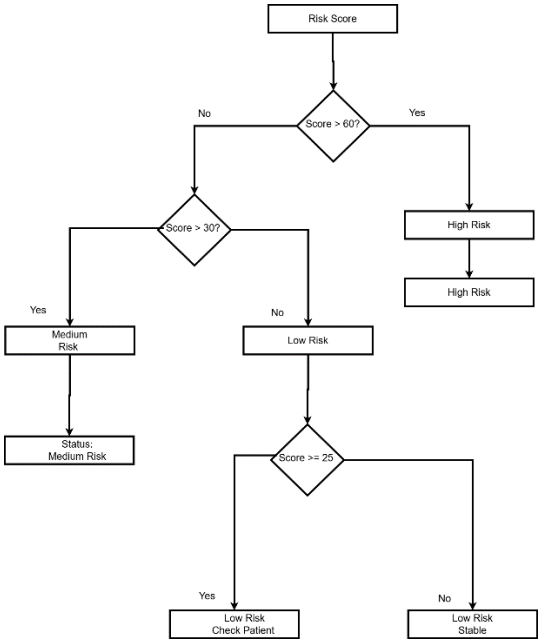


Figure 8: Flowchart for fall detection.

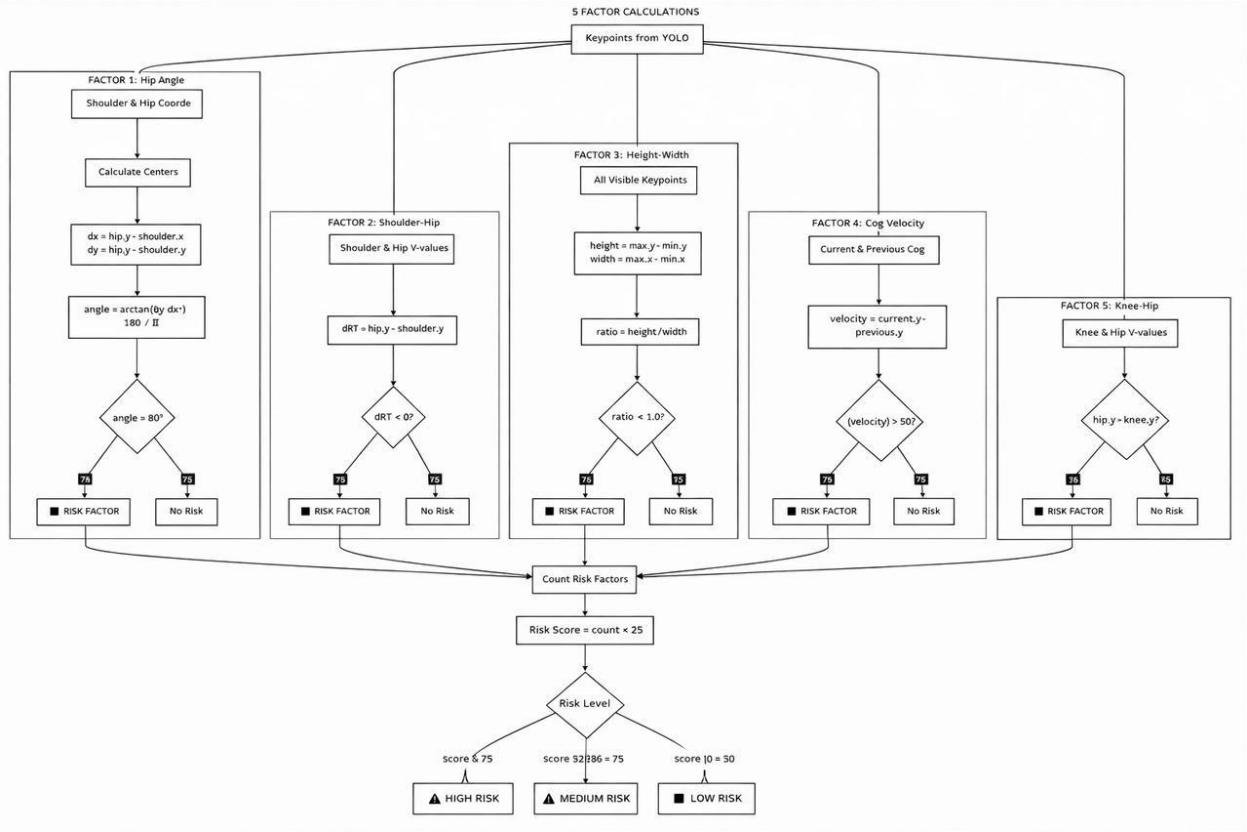


Figure 9: Formulas and calculation thresholds used for predicting the risk factor of patient fall.

3.4.4 Our Approach vs Base Paper Approach of fall detection using basic python:

In this section we will make a comprehensive comparison study from our base research paper. First we have keypoints mapping comparison below in table 6.

Table 6: Showing the comparison for keypoints mapping values.

Body Part	Paper Index	Our Approach	Status
Nose	0	0	D
Left Shoulder	5	5	S
Right Shoulder	6	6	S
Left Hip	11	11	S
Right Hip	12	12	S
Left Knee	13	13	S
Right Knee	14	14	S
Left Ankle	15	Used only in visualization	S
Right Ankle	16	Used only in visualization	S

Below in table 7 we have CoG calculation comparison.

Table 7: Showing the comparison for CoG calculation.

Parameter	Research Paper	Our Approach	Difference
CoG Points	0,5,6,11,12,13,14(7 points)	5,6,11,12(4 points)	Paper used head and knees points additional.
CoG Formula	CoG = (1/N) sum(xi,yi)	Cog_x = (LS.x+RS.x+LH.x+RH.x)/4 Cog_y = (LS.y+RS.y+LH.y+RH.y)/4	Both compute mean, conceptually same
Confidence Threshold	ci>0.5	Conf > 0.5	S

Now we have comparison table 8 for body angle calculation.

Table 8: Showing the comparison for body angle calculation.

Parameter	Research Paper	Our Approach	Difference
Shoulder Center	Shoulder = (P5+P6)/2	S	
Hip Center	Hip = (p11+p12)/2	S	
Body Angle Formula	Body_angle = abs((y_hip-y_shoulder)/(x_hip – x_sholder + e)) Where e=10^-5	Angle_rad = arctan2(abs(dx),abs(dx)) Angle_deg = degree(Angle_rad)	Mathematically different Paper: slope Our approach: angle in degree
Fall condition	Body_angle < 0.5	Body_angle<30 degree	Approx. same
Bent over	0.3 <body_angle<0.6	30 < angle < 45	S
Standing	Body_angle > 0.7	Body_angle > 70 degree	D

Now we have comparison table 9 for hip-shoulder relationship.

Table 9: Showing the comparison for hip-shoulder relationship.

Parameter	Research Paper	Our Approach	Difference
Calculation	Y_hip < y_shoulder - 20 (for fall)	Shoulder_hip_diff = hips_y – shoulder_y	D
Risk Threshold	Y_hips < y_shoulders – 20	Shoulder_hip_diff < 0	D
Interpretation	Hips must be 20px + above shoulders	Any negative value (shoulder below hips)	D

The most important factor is Height/Width ratio as this factor in not present in paper, this is our own inclusion shown in table 10 below.

Table 10: Showing the H/W ratio factor calculation

Parameter	Research Paper	Our Approach	Difference

Formula	No	Height = max(y) – min(y) Width = max(x) – min(x) Ratio = height/width	D
Threshold	No		D

Below we have posture classification threshold comparison table 11.

Table 11: Showing the threshold comparison.

Posture	Research Paper	Our Approach	Difference
Fall detected	Body_angle < 0.5 AND y_hip<y_shoulder -20 AND abs(y_hips – y_feet)	Multiple Factors 1)angle<=30 2)shoulder_diff < 0 3)H/W <=1.0 4)abs(velocity) > 10 5)knee_hip_collapse = True	D
Sitting	Body_angle > 0.6 AND y_hips < y_shoulder	Not included	D
Bent over	0.3 < body_angle < 0.6 AND y_hips > y_feet + 30	30 < angle <=45 = bent	D
Standing	Body_angle > 0.7 AND y_hips > y_shoulders	Angle > 70 = upright	D

Another factor that we added in our approach is CoG velocity calculation to analyze the rapid change in body position. Below table 12 shows about this factor

Table 12: Showing the CoG velocity calculation

Parameter	Research Paper	Our Approach	Difference
CoG Velocity	No include	Velocity = current_y – previous_y Abs(velocity) > 10 px/frame	D
Threshold	No include	>10px/frame	D

Now we will conduct overall algorithm comparison showing in table 13 below.

Table 13: Showing the overall algorithm comparison.

Aspect	Research Paper	Our Approach
Approach	Rule-based (AND condition)	Multi-factor Scoring (25% for each)
Factors used	3 main factors	5 factors + temporal analysis
Threshold Precision	Specific pixel value	Angle degree, ratio , Booleans

Fall definition	Strict AND for 3 conditions	Score-based (>=3 factors = High Risk)
Ankle/feet Usage	More	Less
Posture Classes	Fall, sitting, bent and standing	Upright, leaning, bent and horizontal
Temporal Analysis	Not included	Angle change rate, consecutive frames

3.5 Flowchart for family member recognition module:

Patients with dementia or Alzheimer's disease lose the ability to recognize people from their past, so in this portion of the project we will use facial recognition to assist them in becoming familiar with their family members again. The facial recognition robot identifies and tells patients who that person is and how they are related to patients—the patient's mother, for instance. We will use a camera to identify the person. The robot will use digital image processing techniques to detect their face and extract the features in order to match them with the faces of family members in the database. In case the robot could not find a face, it will continue looking for that person.

Once the facial recognition robot successfully identifies the face, it will activate the Bluetooth speaker so the robot is able to tell the patient how they are related to that person and will begin looking for other faces.

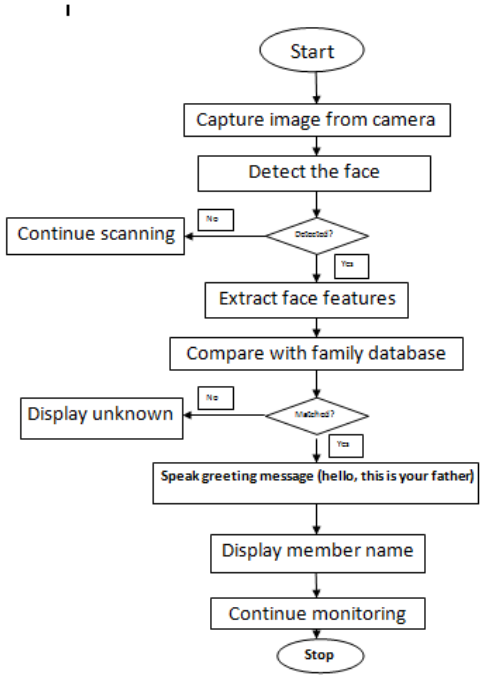


Figure 10: Flowchart for family member's recognition

3.6 Algorithm for Environment Monitoring:

The algorithm in figure 11 illustrates how an environmental monitoring system built around a ESP32 microcontroller monitors both gas and smoke levels using various sensor technologies. The first step in the flow of the algorithm is to initialize the monitoring system and ESP32 controller, followed by powering on all connected sensors (MQ2 for gas detection, temperature sensor, and ultrasonic sensor) and reading the values from each of them in sequence.

The flow of the algorithm continues to the decision nodes of the system to check if the gas level is greater than 300. If the gas level is greater than 300, the system generates an alert that gas has been detected. If not, the system checks if the smoke level is greater than 800 and, if it is, generates an alert that smoke has been detected. If neither of these conditions has been met, the sensor data will be printed to the serial monitor. This process continues until the system is stopped, thereby providing continuous gas and smoke level monitoring while providing alerts immediately when hazardous conditions have been detected. This flow is well defined and logically structured and is designed to provide real-time processing of any data during the monitoring process.

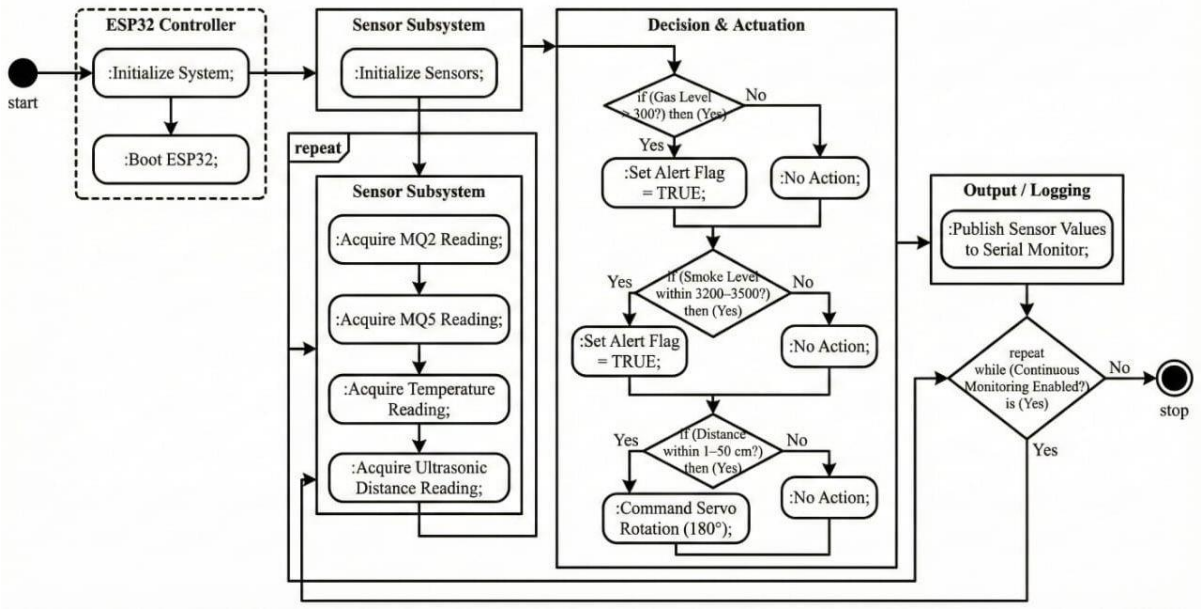


Figure 11: Algorithm for hardware component

3.7 Use case Diagram

Figure 12 depicts an IoT based system for Smart Robotic Assistance for patients suffering from Alzheimer's disease. The IoT- based system interacts with the patient through voice interaction, medication reminders, emergency assistance, patient tracking, and obstacle avoidance. The goal of using these methods is to enhance both the patient's safety and support throughout the day. The IoT system allows the caretaker to monitor the patient remotely and check for live and current activity through a mobile viewer. All of these interactions allow for greater independence and safety for the patient as well as providing the caregiver with real-time visual support through connectivity to the IoT.

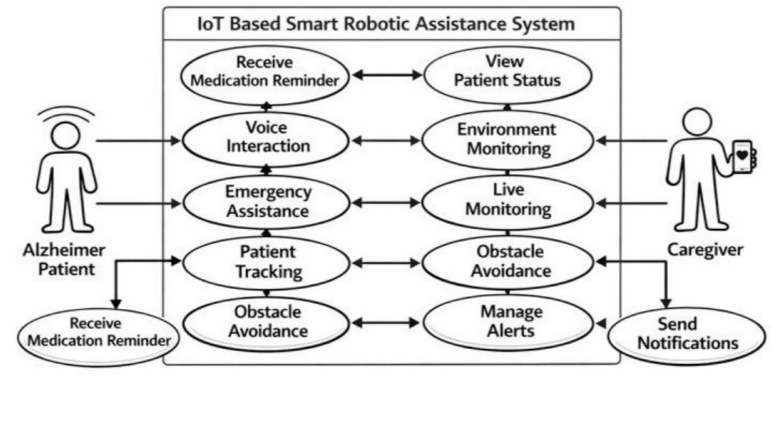


Figure 12: Use case diagram.

Chapter 4
IMPLEMENTATION AND RESULTS

In this chapter we will first describe the components of hardware and software that we are using in order to built this project, then we will start to explain the implementation of each module one by one along with results.

4.1 Hardware Components:

We have used simple and affordable hardware components that are easily available:

4.1.1 Raspberry pi 5/ 4GB:

This is the robot's main central processing unit that performs all AI models and computer vision tasks. It has an interface to the camera, speakers, microphones, motor drives for wheels, and many other devices.

Let's talk about the hardware specifications. The hardware includes an RP1 I/O controller (see figure 14), a Broadcom BCM2712 CPU/Processor - 2.4 GHz Quad-Core 64-Bit ARM Cortex-A74 with Cryptography Extensions (see figure 15), and a VideoCore VII GPU. It has a dual 4K monitor output with HDR Support (see figure 16). This hardware has a maximum of 16 GB of RAM, but only 4 GB of LPDDR4X-4267 SDRAM will be used in this project.

The hardware provides dual-band 802.11ac Wi-Fi and Bluetooth 5.0/Bluetooth Low Energy for Wi-Fi and Bluetooth connectivity. This hardware also comes with a microSD card slot and supports of high-speed SDR104 mode.

The Pi 5 has two USB 3.0 ports and two USB 2.0 ports. The USB 3.0 ports support simultaneous 5gbs operations (see Figure 19). Additionally, Pi 5 also has Gigabit Ethernet support on its motherboard (also see Figure 19). The Pi 5 has two MIPI camera/display interfaces (4-lane) shown in Figure 20. As well, there are PCIe 2.0 x1 interface(s) for high-speed peripheral devices (like SSDs). The Pi Board has an external power button and battery to power the real-time clock. Furthermore, the Pi 5 features a UART debug port (see Figure 17) for serial communications. Additionally, the Pi 5 motherboard features a Type C USB cable the supplies power to the board (see Figure 18). Finally, refer to Figure 21 for GPIO Pins that can be used to connect external peripherals to the Pi 5, such as: motors/drivers, sensors.



Figure 13: Raspberry pi 5 (4 GB) unboxing

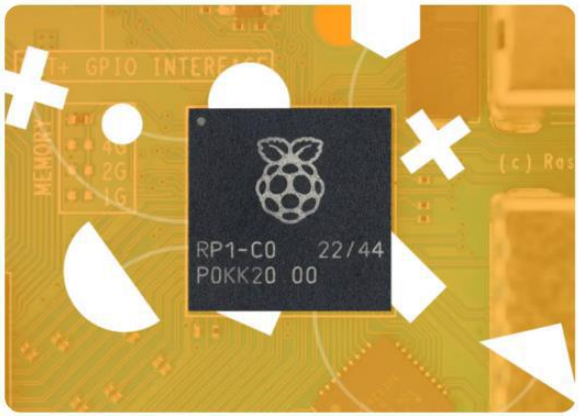


Figure 14: RP1 I/O Controller of raspberry pi 5, 4 GB



Figure 15: Broadcom BCM2712 quad-core Arm Cortex A76 processor @ 2.4GHz



Figure 16: Dual 4Kp60 display ports



Figure 17: UART debug port



Figure 18: C-type port for powering to pi5



Figure 19: USB and Ethernet Port for pi 5



Figure 20: Camera connection ports of pi5



Figure 21: GPIO Pins of pi5

4.1.2 Raspberry pi 5 casing

To protect our module from dust, heat generated in the environment, and other harmful objects, such as filling equipment with moisture and dust, we use the original raspberry pi case shown in figure 22. The raspberry pi case has an integrated fan to help keep the device cool, and the case provides easy access for connecting the GPIO port connections. The white-pink raspbery pi5 case is a great option for keeping our hardware safe while providing optimal/temperate cooling.



Figure 22: White-pink casing of raspberry pi 5 (4 GB)

4.1.3 Raspberry pi5 Power Adapter:

Below in figure 23, we have clear image of pi5 power adapter, this adapter can **vary the voltages from 5v upto 12v as per load on pi 5, this variation keep the pi5 in safe zone while running heavy models on pi5.**



Figure 23: Power adapter of raspberry pi 5

4.1.4 SD Card 128 GB

In this module, we have SD card slot to boot the pi smoothly and fastly in safe zone. Below in figure 24 we have clear view of slot. We flash this card with OS files of pi, then insert card into this , connect power ,display with pi and this card then boot up pi5, without this card pi can't work.



Figure 24: SD Card slot for pi 5 /4gb

4.1.5 Raspberry pi 5 Camera Modules

The raspberry pi camera will be our input device used for taking pictures of humans with the camera v3.1 board we used. The camera board is shown in figure 25. The camera has a ribbon cable rather than a standard wire; the length of the ribbon cable is 20cm with a reasonable width. The camera ribbon will slide into the camera connector (the silver side will face the pi ethernet).

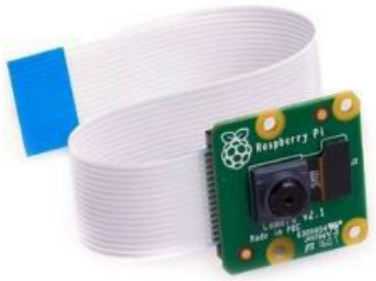


Figure 25: Pi camera v3.1 with ribbon

4.1.6 Esp32 Microcontroller

The esp-32 microcontroller can perform a variety of tasks to monitor the environment (i.e. ambient temperature measurement, gas detection, smoke detection, etc.). The esp-32 has lightweight module options and can be utilized to perform the required tasks. Therefore, we will not require a pi 5 level of processing for environmental monitoring. As a result, we have determined to distribute our workload across multiple hardware devices to maximize our efficiencies, and will use the esp-32 to perform all lightweight modules and the pi5 to perform larger modules (i.e. artificial intelligence and computer vision) jobs.



Figure 26: Esp-32 microcontroller of environmental monitoring

4.1.7 Motor driver modules

To allow for robot mobility we will drive the wheels with motors using motor drivers. The BTS7960 H-Bridge Motor Driver can support up to 12 volts, allowing it to run and also provides a heat sink to keep the motor driver at a low temperature when operating with large tonne robot and battery pack weight. We will use 2 x motor drivers for the left and right motor because this driver can only connect one motor at a time.



Figure 27: BTS7960 Motor Driver

4.1.8 DC Gear Motor

In order to rotate the wheels of the robot we will use a quality DC Gear Motor purchased at 12 volts; supporting 86 rpms and providing sufficient torque for smooth operation of the robot.



Figure 28: DC Gear Motor

4.1.9 Robotic Wheels

Blue rubber wheels will be used with a diameter of 88mm and made from ABS plastic for the constructing the wheel. The surface of the wheel has been treated with vacuum ultraviolet light, resulting in the non-odorous surface of the tires constructed from ABS will have considerably more surface friction.



Figure 29: Blue rubber wheels

Specifications of wheels are below,

- Diameter: 65mm
- Width: 27MM
- Inside corner to corner: 12MM
- Applicable: competition, robotics, distance running, strong grip

4.1.10 Bluetooth speakers

For robot voice, we are using high quality Bluetooth speaker shown in figure 30 below. We will use them for voice alerts and reminders for patient like medication alerts, fall interaction and family member recognition.



Figure 30: Bluetooth speakers

4.1.11 USB microphone

This component will help us get the patient's responses during the interaction on fall detection. Below we can see an image of the mic in figure 31.



Figure 31: Microphone for patient response

4.1.12 12v external battery pack

We are going to use a 12V, 7A battery to make the head of the robot wireless; this battery will provide the voltages necessary for driving the wheels (through the motor driver) and turning on the Pi5 (to allow for wireless operation). Below, in figure 31, we see an image of the battery; this battery is rechargeable.



Figure 32: 12v/7A external rechargeable battery pack

4.1.13 MQ-2 Sensor



Figure 33: MQ-2 Sensor

The MQ-2 gas sensor detects the presence of smoke and combustible gases (like LPG, methane, hydrogen, alcohol vapor and propane) by measuring the change in the electrical resistance of the SnO_2 sensing material when combustion gases or smoke are detected. When the concentration of gas or smoke goes up, the sensor measures the decrease in resistance and converts the resistance change to an analog voltage that the microcontroller can read and use to determine the level of gas or smoke discovered. Some of the more common uses for this sensor would include: smoke detection devices, home and commercial gas leak detection, fire safety systems, and IoT-based monitoring devices that will alert the end-user to take action when dangerous levels of gases or smoke are discovered, as shown in figure 33.

4.1.14 MQ-5 Sensor



Figure 34: MQ-5 Sensor

The MQ-5 gas detection sensor is used to identify gas leaks (natural gas, LPG) or the presence of hydrogen. It has tin dioxide (SnO_2) as a sensing layer; when gas molecules bind with oxygen, electrons flow reducing the resistance of SnO_2, producing an analog voltage.

The production of an analog voltage allows a microcontroller to identify the concentration of combustible gases (as long as a chemical reaction occurs at the SnO_2). Thus, the sensor may be used in numerous settings—home gas leak detection alarms, industry safety systems, automatic

ventilation control, and smart IoT devices that send notifications of leaks or activate safety protocols when a leak occurs (see fig 34)

4.1.15 LM35 Sensor

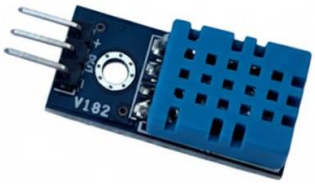


Figure 35: LM35 Temperature Sensor

To measure temperature, the temperature sensor detects change in the physical or electrical characteristics associated with temperature change by means of detecting temperature change, based on type (i.e. thermistor/thermocouple/digital temperature sensor) changing resistance/voltage/generating a digital output corresponding to the measured temperature. A microcontroller receives the output signal from a temperature sensor, and then converts that signal to an exact temperature reading. Examples of real-world applications of temperature sensors can be found in weather stations; home appliances; industrial machinery; medical devices; and smart IoT systems where they are used to help ensure safety, maintain control to heat/cool and monitor environmental conditions as seen in Figure 35.

4.1.16 Ultrasonic Sensor



Figure 36: Ultrasonic Sensor

An ultrasonic sensor measures distance by sending out high-frequency sound waves and calculating the time it takes for the echo to return after hitting an object. Inside the sensor, a transmitter emits ultrasonic pulses, and a receiver detects the reflected waves. The sensor's internal circuitry measures the time difference between sending and receiving the sound, converts it into distance, and sends this information to a microcontroller. In real-life applications, ultrasonic sensors are used for obstacle detection in robots, automatic parking systems in cars, water level monitoring, smart dustbins, and various IoT systems that require accurate non-contact distance

measurement shown in figure 36.

4.1.17 Servo Motor



Figure 37: Servo Motor

A servo motor works by using a small DC motor combined with gears and a built-in feedback system to precisely control angular position. Inside the servo, a control circuit constantly compares the motor's current shaft angle with the desired angle sent as a control signal (usually a PWM pulse). If the shaft is not in the correct position, the motor automatically rotates in the required direction until the target angle is reached. In real-life applications, servo motors are commonly used in robotic arms, drone controls, RC cars, manufacturing machines, and smart automation systems where precise, accurate, and repeatable movement is required shown in figure 37.

4.1.18 Vero-Board and soldering Material

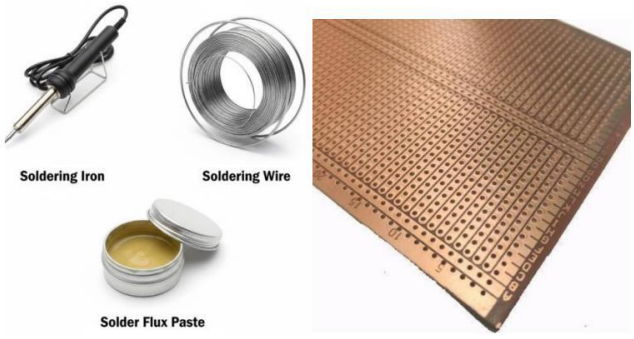


Figure 38: Soldering Material and Vero-Board

A soldering iron is a small tool that can be used to join electrical parts together by melting solder. When you turn on the power, the tip, which is usually made of copper, gets very hot (between 250°C and 400°C) to melt the solder. To join two or more parts together, you take the soldering iron and put the hot tip in contact with the joint of the wire or lead on the PCB, then add the solder wire to the hot soldering iron tip so that the solder melts and flows around the entire connection

and creates a shinny metal connection. Once you take the soldering iron away from the connection, the solder cools and solidifies; thus, creating a strong and reliable electrical and mechanical connection. When using a soldering iron for production or general use, you need to keep the soldering iron tip clean using either a damp sponge or brass wool, ensure that you have the correct soldering iron temperature, and avoid touching the hot soldering iron tip with your fingers as this could lead to burns at the location indicated in figure 38.

4.1.19 Bread Board and Connecting Wires

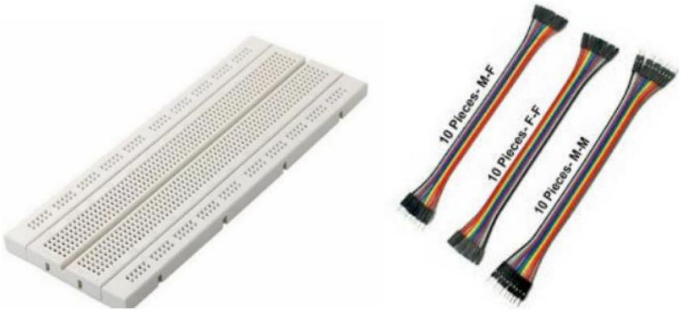


Figure 39: Breadboard and Connecting Wires

The Breadboard is used for testing the components, analyze the operations of hardware and for this purpose we need connections so connecting wires will be used for making the connections shown in figure 39.

4.2 Explanation of Components Grouped by Features

4.2.1 Fall Detection



Figure 40: Grouped components for fall detection

To facilitate the detection of falls, we incorporate a CSI camera, microphone. speakers, and a

Raspberry Pi 5 with 4GB of RAM. The CSI camera captures real-time video, enabling the Fall Detector to determine falls through visual cues. The speakers audibly notify the victim/user or caregiver of the fall. The microphone detects sounds associated with falling and improves detection. Together, these components are all connected to and powered by the Raspberry Pi 5 with 4GB of RAM, which processes data quickly for real-time response, as shown in Figure 40.

4.2.2 Patient Tracking



Figure 41: Grouped components for patient tracking

For patient tracking, we use RC tires, a CSI camera, Raspberry Pi 5 with 4GB, a battery, jumper, and motor driver. The RC tires allow the tracking device to move around freely while following a patient. The CSI camera captures real-time video to monitor the patient's surrounding area. The Jetson Nano processes the data from the camera and sensors to ensure accurate tracking. The battery powers the system, and the motor driver controls the RC tire movement, allowing for smooth movement during the tracking process, as shown in Figure 41.

4.2.3 Family Members Recognition



Figure 42: Grouped component for family member's recognition module

Using a camera for facial recognition and a Raspberry Pi 5 to process this facial recognition along with a speaker for communicating the return on investment between a client and his/her/its partner will all be powered by the power supply shown in figure 42.

4.2.4 Temperature Sensing

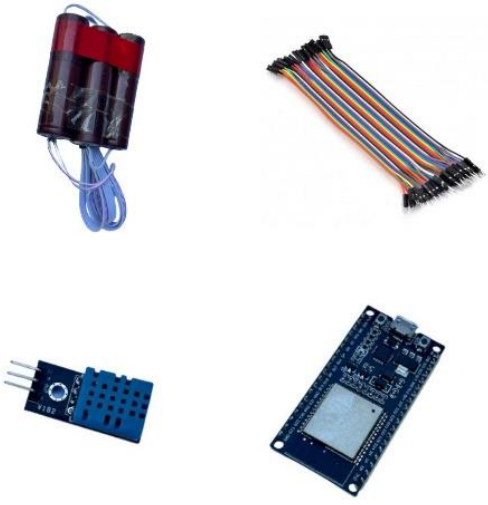


Figure 43: Grouped components for temperature sensing.

Similarly, for temperature monitoring, the temperature sensor is connected to the ESP32 via jumper wires whereby the DHT11 temperature sensor detects the surrounding temperature and

transmits this information to the ESP32 for processing. The ESP32 processes the information received and sends the temperature information back to the DHT11 summary, enabling the DHT11 to give real-time temperature measurements along with a data transmission display via the jumper wires. Examples of this are also shown in figure 43.

4.2.5 Gas Sensing

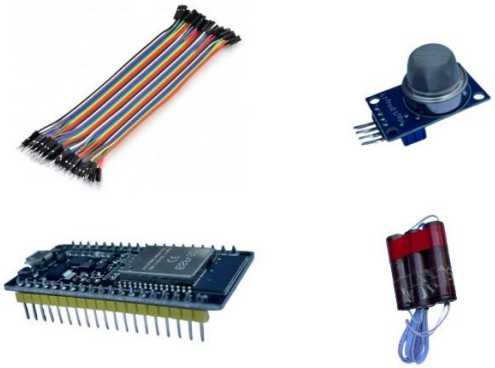


Figure 44: Grouped components for gas sensing

For the gas detection system, an MQ5 sensor, battery, ESP32, and jumper wires are used together whereby the MQ5 sensor detects gas concentrations within its environment and transmits this information to the ESP32 for processing. Once the ESP32 has processed the information received from the MQ5, it will send out all necessary notifications and/or actions based on its internal programming. Jumper wires are used to connect the components of the system with each other to ensure proper communication between the gas detection sensor, the microcontroller (ESP32), and the power supply.

4.3 Simulations and Results

4.3.1 Gas and Smoke Sensing

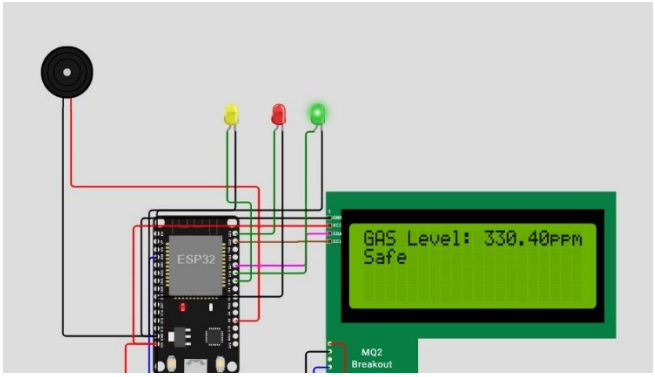


Figure 45: Implementation and result of smoke and gas sensor on simulator

An ESP32 microcontroller, an MQ2 gas sensor, and an LCD display are included in the circuit shown in Figure 45. The MQ2 Sensor detects gas levels, while the ESP32 processes the Sensor data and provides it on an LCD. There are LED indicators (green, yellow, and red) for both visual indications of gases in the area, such as "Safe" or "Dangerous." The LCD displays the gas level (for example, 330.40 ppm) and a Warning/Caution message. Monitoring the level of gases in the area will give an indication if an unsafe gas level is present.

4.3.2 Temperature Sensing

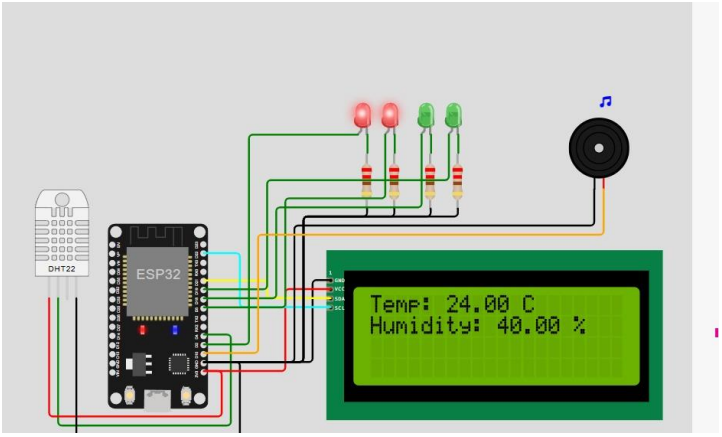


Figure 46: Implementation and result of temperature sensor on simulator

A DHT22 temperature/humidity sensor, an ESP32 microcontroller, and an LCD can be seen in the circuit in Figure 46. Similarly, the DHT22 Sensor measures the temperature and humidity and displays it on the LCD. There are also LED indicators (Red and Green) and a buzzer when the temperature and humidity levels reached a certain threshold. The LCD shows the current temperature of the environment (for example, 24.00 degrees C) and current humidity (for example, 40.00%) to provide clear Monitoring of environmental conditions.

4.3.3 Distance Measuring

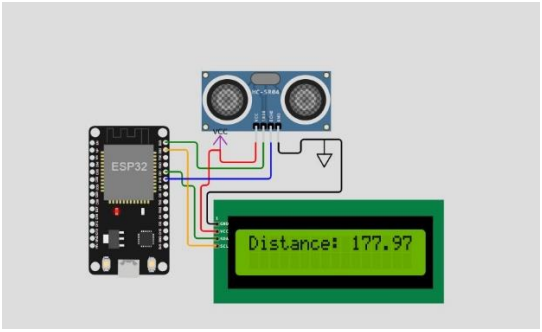


Figure 47: Implementation and result of ultrasonic sensor on simulator

Figure 47 shows a circuit setup with an ESP32 microcontroller, an HC-SR04 ultrasonic sensor, and an LCD display. The HC-SR04 ultrasonic sensor measures the distance to an object by sending out sound waves and calculating the time it takes for the sound to bounce back. The ESP32 processes this data and displays the calculated distance (e.g., 177.97 cm) on the LCD screen. The setup allows for real-time distance measurement, making it suitable for applications like object detection or proximity sensing.

4.4 Breadboard Implementation and Results:

4.4.1 Smoke Detection

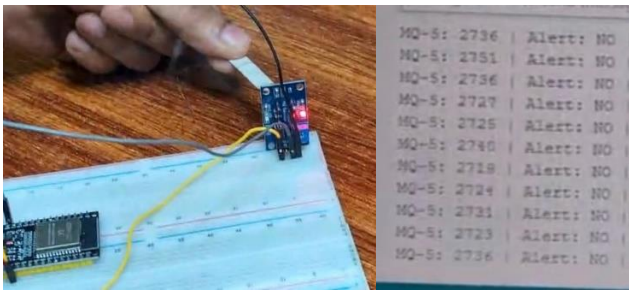


Figure 48: Implementation and result of smoke sensor on breadboard

The MQ-5 smoke detection sensor was monitored with the ESP32. The range for a smoke level that indicates an alarm is when the concentration level reads between 3200-3500, at which time the system will indicate the status "Yes" for alerting you to smoke. Any concentration level less than 3200 will result in a "No" status indicating that your smoke concentration is safe. Therefore, this is a very effective means of monitoring smoke levels in your environment shown below in figure 48.

4.4.2 Gas Sensing

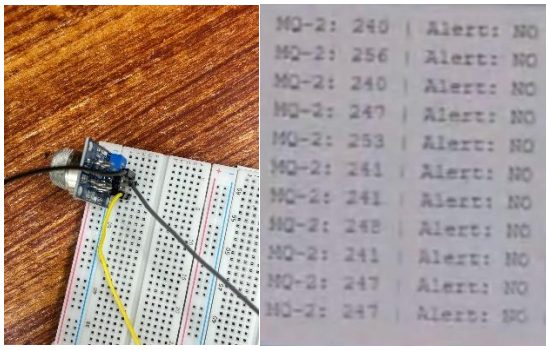


Figure 49: Implementation and result of gas sensor on breadboard

The ESP32 monitored the MQ-2 gas detection sensor. A threshold gas concentration value was

established at approximately 300, above which an alert status of "Yes" will occur indicating that there is a gas hazard present. Any gas concentration reading lower than the threshold indicates that there is no hazard for gas. Accordingly, this is an effective method for determining the difference between normal and hazardous gas concentrations illustrated below in figure 49.

4.4.3 Distance measuring and Obstacle Detection

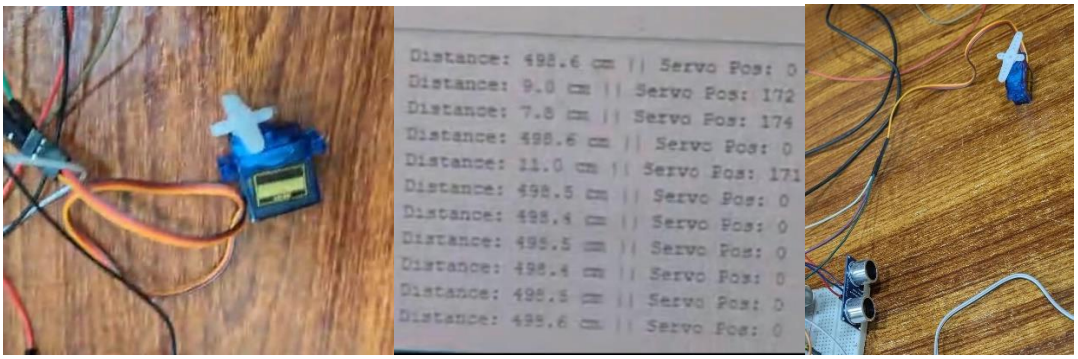


Figure 50: Implementation and result of ultrasonic sensor on breadboard with servo motor.

For obstacle detection, the HC-SR04 ultrasonic sensor was used to measure distances from objects. The ESP32 continuously monitored the distance values. When an obstacle was detected within the range of 1 to 50 cm, the servo motor was activated to rotate 180 degrees, helping the system avoid the obstacle. Distances beyond this range did not trigger any movement. This setup ensures effective obstacle avoidance using real-time sensor data shown in figure 50.

4.5 Software models/libraries:

4.5.1 Raspberry pi OS installation and Result

To work with the Raspberry Pi 5, you first need to install the appropriate operating system on an SD Card. The following outlines how to prepare a 120 GB SD Card to install the Operating System.

- Start by inserting the SD Card into a memory card reader or the laptop's SD Card slot (depending on which hardware you have). Format the SD Card so it can be flashed.

- Download and install the Pi Imager to flash the Pi with an image of the Operating System (refer to the image in Figure 51).

- Choose the appropriate version of the Raspberry Pi by selecting "Raspberry Pi 5" (as shown in image 52), then select the version of the Operating System you will be using. The Raspberry Pi 5 will use the 64-Bit Operating System.

- Specify the SD Card has been inserted and has been prepared as an SD Card to be used with the Raspberry Pi 5.

- Enter a name for your Raspberry Pi that will be unique on the Network that it will be connected to, so that you can connect remotely to it in the future (see Image in 53).

- Set your Raspberry Pi's time zone and date accurately to your location by using the Pi Imager to do this at the time of flashing your Raspberry Pi (refer to the image in Figure 54).

- Choose a default username and password for pi5.

- Connect pi to an available network, using this network also connects to the same network you connected your personal device to so you are able to view pi5's graphical user interface (GUI) on your personal device, inputting SSID name and password when prompted, and selecting "Next" when complete, as shown in figure 55 below.

- Enable SSH access so you can remotely access pi terminal as seen in figure 56 below.

- Enable raspberry pi connection for graphical user interface access remotely, without monitor or any peripheral devices attached.

- Authenticate connection to pi by entering authentication key in textbox.

- Write the operating system image file (OS) to SD card.In figure 57, you can see our first bootup view of pi5.



Figure 51: Raspberry pi 5 imager view

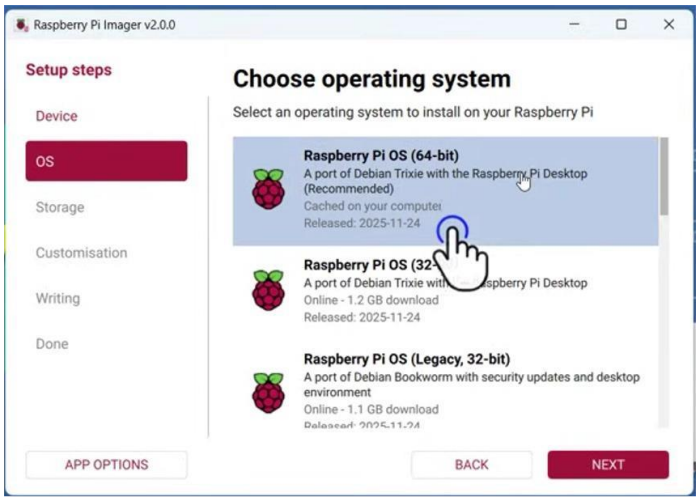


Figure 52: OS version selection

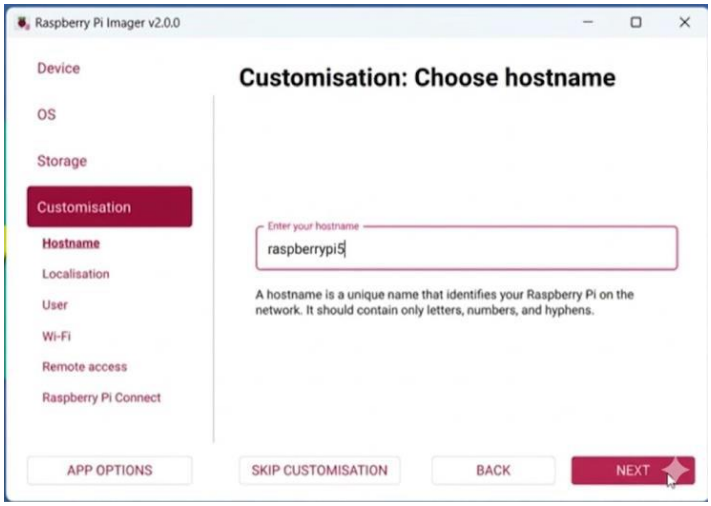


Figure 53: Hostname of pi 5.

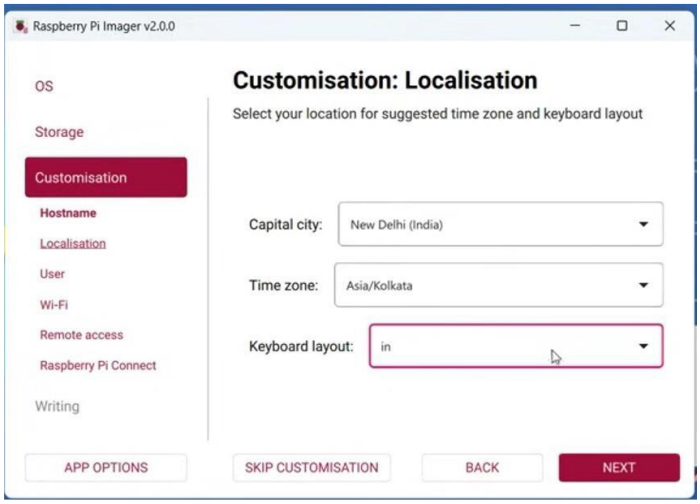


Figure 54: Select location and time zone of pi5.

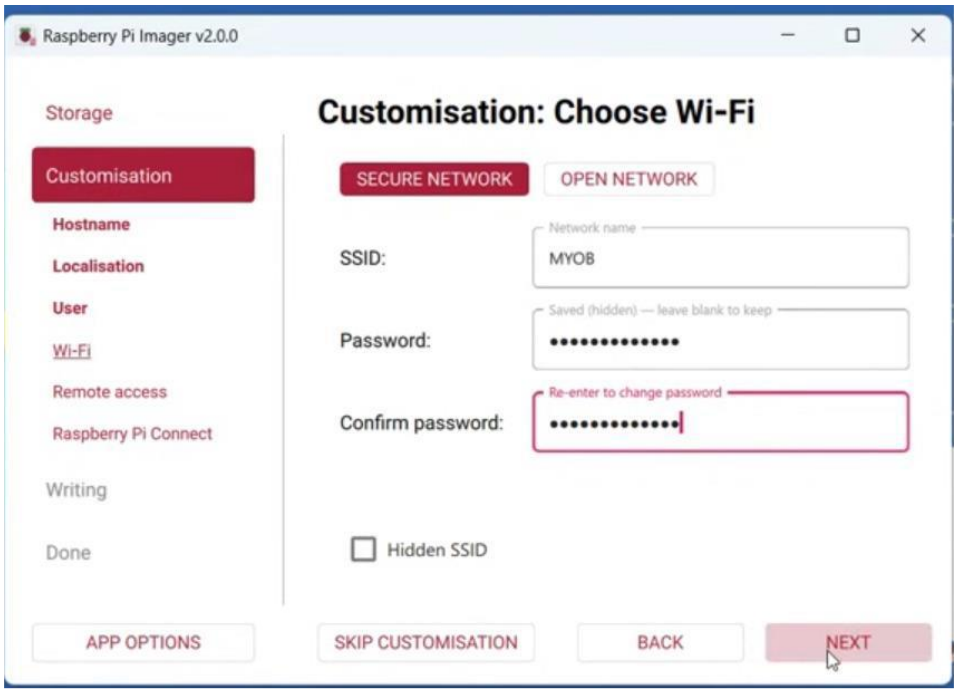


Figure 55: Internet setup of pi5.

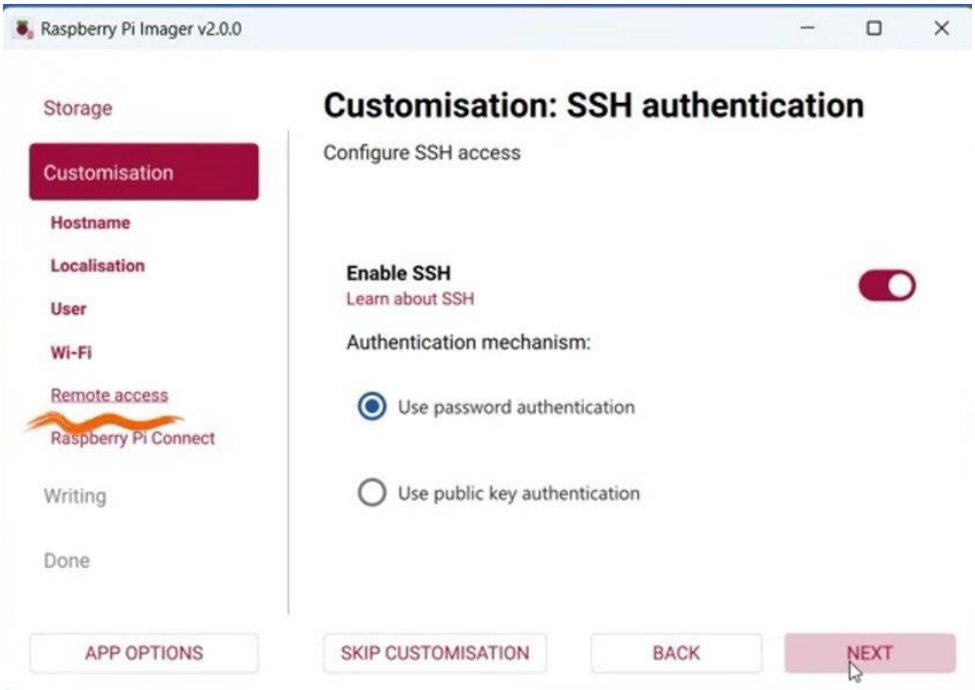


Figure 56: SSH enabling of pi 5

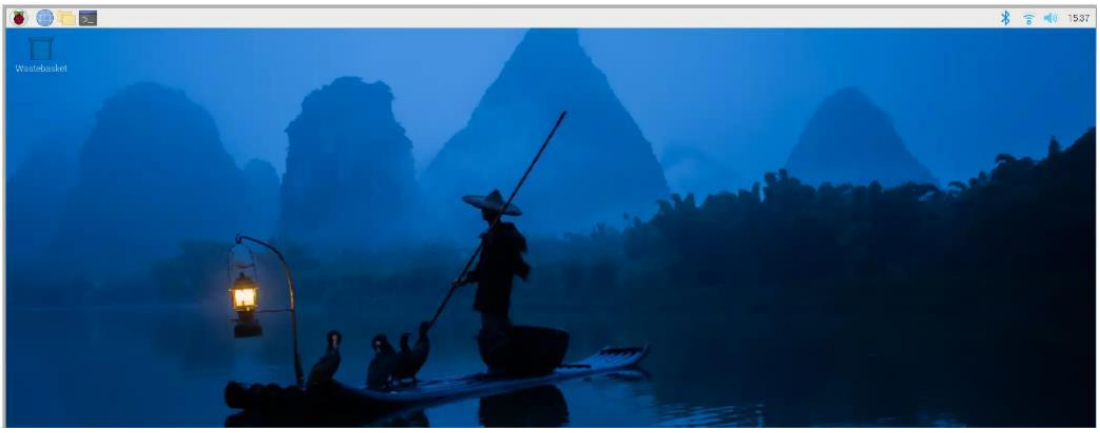


Figure 57: Raspberry pi 5/4GB OS installation and desktop interface after bootup.

4.5.2 Python Language (Primary use)

Python programming is used for implementing each module like fall detection, tracking/following, family member recognition and environmental monitoring.

4.5.3 Yolon8n-pose Model

We are using **YOLO Family models to** perform computer vision tasks, for fall detection we have to analyze human body posture, so that we will use **yolov8n-pose model,** this model detects the human body in frame along the keypoints, keypoints are points on body shown in figure 58 below. We will apply conditions on the placement of these points in frame so that we confirm the posture of human/patient.



Index	Key point
0	Nose
1	Left-eye
2	Right-eye
3	Left-ear
4	Right-ear
5	Left-shoulder
6	Right-shoulder
7	Left-elbow
8	Right-elbow
9	Left-wrist
10	Right-wrist
11	Left-hip
12	Right-hip
13	Left-knee
14	Right-knee
15	Left-ankle
16	Right-ankle

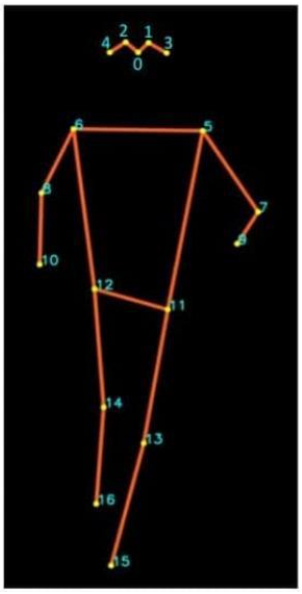


Figure 58: 17 key points detection using yolov8n-pose model

4.5.4 Yolov8n Object Detection model

The yolov8n model has been chosen for tracking the patient. With this approach, we can continuously monitor patients/humans. The fall detection uses a human pose model to determine when an individual is standing or sitting; however, this project does not require the determination of a person's position; rather, it will only need to identify whether or not they are present in the image frame. Consequently, YOLOv8n has been selected as the model of choice rather than a pose model. After the presence of an individual is detected by YOLO, a bounding box is drawn around them, as illustrated in figure 59. Tracking will be accomplished using this bounding box; further detail on the implementation will be included in the next report.



Figure 59: Yolov8n human detection and bounding box

4.5.5 Opencv

OpenCV is known as "Open Computer Vision". This can facilitate image processing and visualizations along with cameras and videos. It provides us with camera frames and overlays to the frames when displaying the frames.

4.5.6 Firebase Realtime Database

We have to alert caregivers of patient room temperature, gas detection, and smoke; caregivers will

monitor all data from the sensors. To move this data from the hardware to the app we will use Firebase to transport the data from the hardware to the Firebase Cloud and then to the app.

4.5.7 Vosk Speech Recognition

Vosk is an offline speech recognition engine that we will use to confirm whether a patient has fallen by providing a voice interaction between the robot and the patient. After the robot has asked the patient for help, the robot will receive a response from the patient and convert the patient's spoken response to text so that the robot understands how to respond by using the Vosk Speech Recognition Offline Engine.

4.5.8 eSpeak Text-to-Speech Engine

Now for asking robot to patient we will convert text into speech for Bluetooth speakers. What we will do ? we will use eSpeak and then send our text to this model this will generate voice of that text and play it in speakers.

4.5.9 Picamera2 Library

We will install and use this library to interface with pi camera v3.1 board to capture live streaming of patient.

4.5.10 Numpy

We have used numpy for numerical calculations and mathematical computations while predicting the fall and tracking the human. For risk factor calculation, for predicting place of keypoint in frame in pixels we will use this library.

4.5.11 GPIO Library

We will install this library into pi5 for controlling motor driver pins and other hardware interfaces.

4.6 Modules implementation

4.6.1 Patient Tracking Module implementation

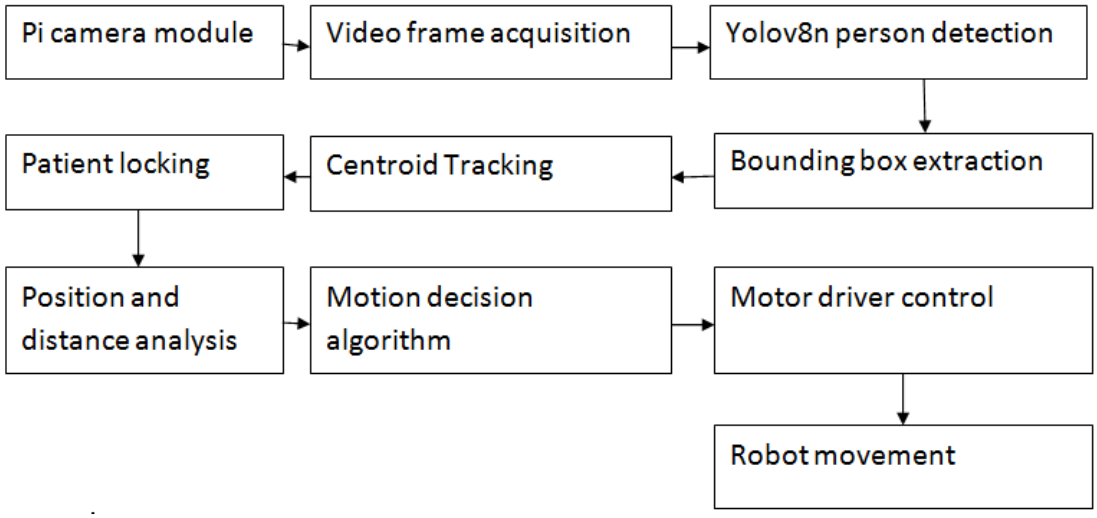


Figure 60: Architecture of patient tracking module.

The implementation of this module will start with the camera input using a pi camera v3.1 to capture images of patients, and since the video is simply a collection of images taken sequentially over time, the images can be extracted from the video, and the yolon8n methodology will be applied to detect humans in every four frames of video. As we mentioned previously, yolov8n generates a bounding box around the human so after detecting the human we now have a bounding box that we can apply to implement the remaining logic required to track the movement of the patient.

After detecting the human, the next step in tracking will be to apply centroid tracking, this will allow us to establish the middle point of the bounding box, and from this center point we can make predictions about where to locate the human in the frame (either to the left, right or in the center). This allows us to then split the frame into the five zones as shown in Table 14 below.

Table 14: Showing the screen distribution for tracking the human motion

Zone	Position	Command	Turn strength
Left most	0-20%	Strong right	0.85
Left	20-35%	Gentle right	0.35
Center	35-65%	Forward /stop/backward	--
Right	65-80%	Gentle left	0.35
Right most	80-100%	Strong left	0.85

Note: LEFT/RIGHT are mirrored, patient on left side of frame means robot turns right to center them.

In order to avoid immediate turn of robot, we keep this turn smooth by dividing frame into 5 zones, based on parameters shown in table 14 above , robot will navigate. **For forward and backward we will apply condition on bounding box length.** If the height is large means person is too much close to robot so robot have to stop or move backward, and if the bounding box height is less means person is far away so robot will move forward. Now we have threshold values of this behavior in table 15 below.

Table 15: Threshold values for forward/stop/backward motion

Box height	Command
<500px	Forward (speed = 28)
500 - 510	Stop
>510 px	Backward

Now we also have **safety function blocks.** If the patient will lost the frame and now robot have to person to follow up , robot will speak in speaker as patient lost, for this we will perform text to speech from eSpeak engine. Next is if the multiple persons will come to frame now robot is

confused with patient so robot will stop and wait for one person to be left in frame to track. Third one if fall is detected then robot will immediately stop and wait for resume motion of patient.

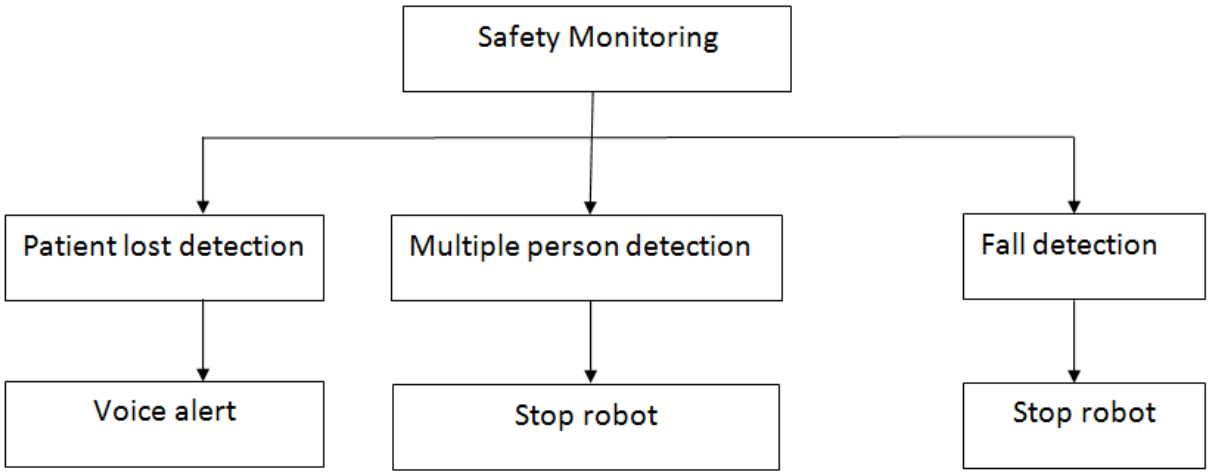


Figure 61: Safety functions block of patient tracking

4.6.2 Patient fall detection module implementation

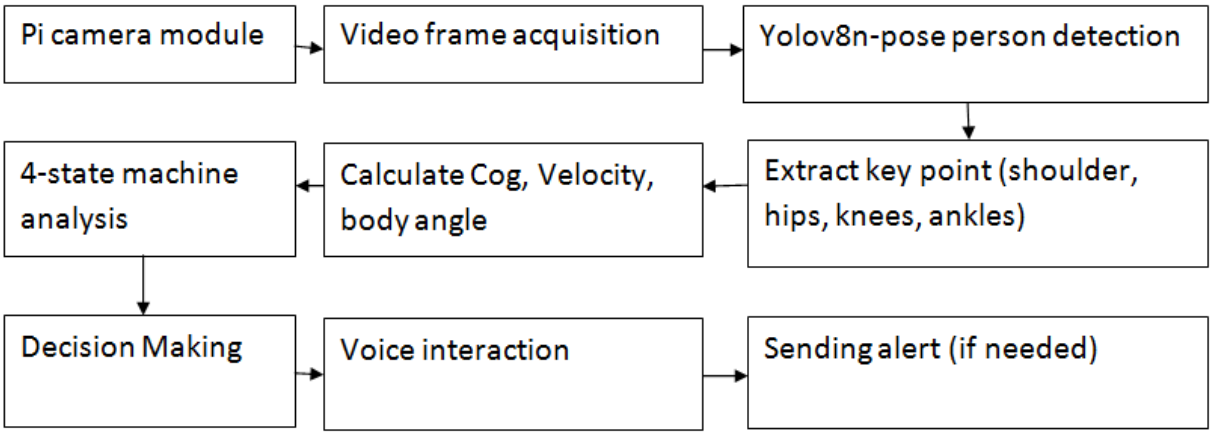


Figure 62: Architecture of fall detection

Now we will start discussing each block in detail in implementation of fall detection. First we will use camera for patient capturing, then we will extract the frames from vides as we did in tracking till here, now we will apply yolov8n-pose for fall prediction analysis.

Following the results obtained from the yolov8n-pose, we have been able to produce a human detection bounding box along with the key points that represent shoulders, hips, knees, and ankles. The next step is to extract the specific key points that relate to the locations of these body segments

and then find the center of gravity - the center point of the human bounding box. After calculating the center of the bounding box, we will determine the speed at which a person is moving based on measurements of the knee, hip, and shoulder key points; and from there, we will also determine the angle (knee-hip-shoulder) that the person's body is traveling at. After completing these tasks, we can move on to decision-making.

We predict falls and make decisions using four states:

- The first state is the "Upright" state, which means the person is standing. If the person is standing, we continue to monitor their stance; if they are not standing, we set that person to be in a "Motion" state.

- If the person is in motion, there is an increase in the rate of speed, and their torso is horizontal to the ground, we will classify this as having registered as a fall; if the person is not in motion, we will classify them as sleeping.

- At one point in time, if all of the preceding conditions are true, we will continue to monitor those same conditions for approximately 15 frames. If all conditions remain true after the preceding 15 frames, we then classify that person as having fallen.

- If the person is indeed falling, before sending an alert to that person's caregiver, we will ask, "Are you okay or do you need any assistance from me?" If the answer is yes, we will record the patient's response via a microphone and then upload that information to Firebase to denote that we have registered an emergency for the patient. If the patient's answer is no, we will also record that and will not send an alert to that person's caregiver.

- If the patient does not respond to the previous question for five or more attempts, the robot will send an alert to that person's caregiver.

Now we will elaborate firebase section that we will send alert on fire base, for that will use .json file and .admin fine and link of firebase in our code along with path where we have to send alert. These all parameters will help us to send alert on firebase. And then from firebase we will send alert to the app.

4.6.3 Family member recognition module implementation

Alzheimer's patients experience significant memory loss, particularly in the early stages of the disease, to the point of not being able to remember people who were previously very close to them and having difficulty recognizing these same people when they meet up again. Therefore, we have developed a module to help support them during the time of their progressions. The following is the architecture diagram for this module.

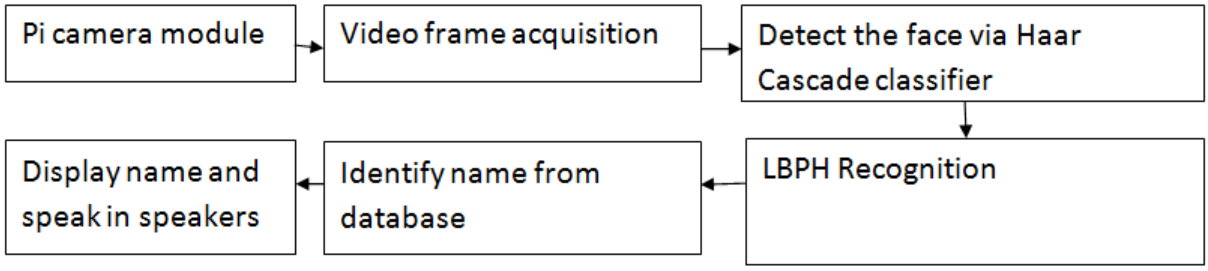


Figure 63: Architecture of family member's recognition module

First we capture human face in frame through face detector, then we will apply LBPH recognition of detected face, this will extract the face features and match them with faces stored in database of family members. Now after successfully matching robot will speak in speakers the name and relationship of person with patient. Robot will announce this name for 3 time and then stop.

We performed this in 5 phases

Phase 1 Live video from the camera of the raspberry pi will be captured while being continuously captured and later converted to grey-level images for quick processing.

Phase 2 Each frame will have the Haar Cascade Classifier utilized to locate faces within the frame. Each face will then be surrounded by a bounding box that will indicate where that face is in the image.

Phase 3 Each detected face will be resized to 200x200 pixels square and an LBPH (Local Binary Pattern Histogram) algorithm will be applied to identify each face against its corresponding face in the database and provide a recognition score.

Phase 4 Identity will be verified by comparing the confidence level against a pre-defined threshold (70). If the confidence level is above the threshold then the face will be classified as a recognized

family member. Conversely if the confidence level is below the threshold, the face will be classified as an unknown person.

Phase 5 After identifying the person as a recognized family member or an unknown, the robot will retrieve the associated relationship-type for the recognized family member, generate a custom greeting using text-to-speech and eSpeak model to greet him/her. The maximum amount of times a family member can be greeted will be limited to three Medication Alerts module implementation.

4.6.4 Medication Alert module implementation:

In this module, we have to retrieve the data from firebase, as caregiver will set time and medication name from **App** then this data will be sent to **firebase** (the cloud) then from firebase, **pi5** will get **data as time and name of scheduled medication alert,** then we will compare the current time with scheduled time, if time matches then we will perform **text-to-speech** conversion and robot will speak through speakers connected with it to say "take your medicine, it's time to take panadol".

For comparing the scheduled time with actual time, we will calculate the difference, if value is 0 then alert generate suddenly, if not then wait for being it 0. The **pi 5** will check **firebase after each 30 seconds** to get data and perform analysis on it.

4.6.5 App Development, Integration with Hardware via Firebase module implementation:

This document re-creates an entire end-to-end IoT data pipeline; the process of collecting sensor data with an ESP32 microcontroller (or Arduino) and streaming that data to a cloud Firebase Realtime Database, followed by the syncing and displaying of that data via an Android app.

This document walks through every step of the project, including: creating code on the ESP32/Arduino using the IDE, configuring sensor communications using ADC/I2C/SPI/UART, leveraging libraries, managing memory/RTOS, debugging, and energy consumption; implementing the Firebase Realtime Database (RTDB) security rules, authentication for Arduino SDK vs REST; creating a data schema to be used with the Firebase RTDB, understanding JSON and latency characteristics of data; integrating Firebase into the Android application through Firebase listeners for push notifications, handling offline syncing, creating data models; data

binding and implementing the MVVM architectural pattern in constructing the application with proper UI pages.

- **Arduino/ESP32 Development:**

Arduino IDE Workflow: The Arduino IDE compiles and uploads C++ sketches to the ESP32 over USB. The user selects the ESP32 board and COM port, then clicks “Verify” to compile and “Upload” to flash (using the underlying esptool)[1][2]. On upload, IDE output shows compilation/linker messages, then binary flashing and reset. The Serial Monitor (baudspecified in Serial.begin()) lets you view debug prints from Serial.println().

Sensor Interfacing: The ESP32 can read sensors via multiple interfaces:

ADC: Use analogRead(pin) to sample an analog voltage (0–3.3V). By default ESP32 ADC is 12-bit (0–4095) resolution[3]. Example: int raw = analogRead(34); // read ADC on GPIO34 float volts = analogReadMilliVolts(34); // get calibrated mV One can set resolution (analogReadResolution(bits)) and input attenuation (analogSetAttenuation()) to scale input range[4]. Each channel’s range varies by chip (see Espressif ADC docs)[5]. ADC is useful for potentiometers, battery level, etc.

I2C (Wire): I2C (Two-Wire) lets multiple sensors share SDA/SCL lines[6]. On generic ESP32, default I2C pins are GPIO21 (SDA) and GPIO22 (SCL)[7]. Usage pattern: #include Wire.begin(); // join I2C bus (master mode).

Wire.beginTransmission(addr);

Wire.write(register);

Wire.endTransmission();

Wire.requestFrom(addr, N);

while (Wire.available()) { val = Wire.read(); }

(Example for a BME280 sensor: after Wire.begin(), call Wire.beginTransmission(0x76);, write register 0xF7, endTransmission, then Wire.requestFrom(0x76, 8) and Wire.readBytes(buf,8) to get temperature/pressure data.) I2C devices need pull-ups on SDA/SCL[8]. Common Arduino libraries (e.g. Adafruit_Sensor, Adafruit_BME280) wrap this **SPI:** Use SPI.h for high-speed peripherals (e.g. SPI ADCs, displays). Initialize with SPI.begin(sck, miso, mosi, ss). Typical usage:

SPI.begin();

digitalWrite(SS, LOW);

uint16_t data = SPI.transfer16(cmd);

digitalWrite(SS, HIGH);

The default VSPI pins on ESP32 (GPIO18 SCK, 19 MISO, 23 MOSI) are used unless reconfigured. SPI settings (mode, speed) can be set with SPI.beginTransaction(SPISettings(8000000, MSBFIRST, SPI_MODE0)).

UART: The ESP32 has multiple UARTs. The primary Serial (UART0) is for programming and Serial Monitor. Additional Serial1, Serial2 can be mapped to other pins[9]. Usage:

Serial1.begin(9600, SERIAL_8N1, RX1, TX1);

if (Serial1.available()) { char c = Serial1.read(); }

UART is used for GPS modules, other MCUs, or debugging. Baudrates up to 5 Mbaud are supported[10].

Libraries & Code Examples: Common libraries include:

WiFi.h (ESP32 core v2.x): connects to Wi-Fi.

Example:

WiFi.begin("SSID","password");

while (WiFi.status()!=WL_CONNECTED)

delay(500);

Firebase_ESP_Client.h (Mobizt FirebaseArduino v2.x): official Arduino library for Firebase RTDB. Provides Firebase.begin(), Firebase.RTDB.setInt(), getFloat(), etc. (Example below)

ArduinoJson (v6.x): JSON parsing/serialization if using REST or custom payloads

Wire.h, SPI.h: as above, built into ESP32 core for I2C/SPI

Adafruit_Sensor.h, Adafruit_.h: e.g. BME280, DHT11

Memory & RTOS Constraints: The ESP32 has ~520 KB SRAM (plus optional PSRAM) and flash (up to 16+MB depending on module)[11]. Under Arduino, it runs FreeRTOS; loop() and other tasks share the CPU cores[12]. Default tasks have large stacks (often 10 KB) but you can reduce them by measuring stack high water marks[13]. Each FreeRTOS task (sensor read, WiFi) has its own stack memory. Heap fragmentation and large library usage (JSON, SSL) must be managed carefully. Typical ESP32 uses ~20–80 mA in normal Wi-Fi operation (depending on TX power)[14].

Debugging: Primary debugging is via Serial.println() logs (Serial Monitor). For crashes (Guru Meditation errors), use the ESP Exception Decoder plugin to interpret stack traces[15]. In Arduino IDE, enable verbose output, use esp_task_wdt.h to detect watchdog resets, or attach a JTAG debugger. For logic issues, toggle LEDs or use breakpoints if supported.

Power Considerations: For battery-powered projects, use ESP32 sleep modes. In modemsleep/light-sleep, Wi-Fi/CPU idle; in deep-sleep, almost everything sleeps. Measured deep-sleep current is ~8 μA (~26.85 μW)[14] versus ~24 mA (~78 mW) when active. Timer or RTC can wake the ESP32 periodically. Remember to disable unused peripherals and use light sleep if fast wakeup is needed.

4.6.6 Firebase Realtime Database Integration:

Firebase Setup & Auth: Create a Firebase project and enable Realtime Database. Secure rules default to authenticated users only. Common auth methods are email/password, anonymous, or using a database secret (legacy) for headless devices. The Arduino Firebase library supports various auth: e.g. using Firebase.begin(&config,&auth) with email/password or tokens, or the simpler legacy mode Firebase.begin(databaseURL, databaseSecret)[16]. (Note: database secrets are deprecated but still work.)

SDK vs REST: You can send data by HTTP REST or via the library. The library handles HTTPS under the hood using WiFiClientSecure. For REST, one would use WiFiClientSecure or HTTPClient to PUT/GET JSON at https://PROJECT.firebaseio.com/path.json?auth=. Using the Firebase_ESP_Client library is simpler.

Data Schema & JSON Format: Firebase RTDB stores data as JSON. Design it to be as flat as reasonable (no deeply nested lists) for scalability Here readingA, readingB could be push IDs or timestamps (e.g. Unix time). Each reading object can include multiple fields. Using push IDs (Firebase.RTDB.push*()) avoids key collisions for simultaneous writes

Security Rules: By default, Realtime DB only allows authenticated access. Rules are JSON on the console.This permits only signed-in users to read/write. For prototyping you might set ".read": true, ".write": true, but this is insecure. Always tailor rules to your needs (e.g. restrict to a certain UID or require write validation).

Latency and Real-Time Sync: Firebase RTDB is designed for low-latency updates. Data writes from the ESP32 arrive quickly on the server, and clients (Android app) listening on that path are pushed updates in ~tens of milliseconds (depends on network). Offline behavior: The Android Firebase client caches active data locally. Writes are applied locally immediately and then synchronized to the server when online. This ensures the app remains responsive even if connectivity drops (local events fire instantly, then server sync).

4.6.7 Data Flow to Android App:

Realtime Listeners: In the Android app, use the Firebase Realtime Database SDK. Add dependency com.google.firebase:firebase-database-ktx and initialize in Application. To listen for data, obtain a DatabaseReference, then attach a ValueEventListener..This listener fires

initially and on every change. Inside onDataChange, you map snapshot to your data model. If using custom data classes (e.g. data class SensorReading(val timestamp: Long = 0, val value: Double = 0.0)), you can do snapshot.getValue().

Offline Sync: Enable disk persistence in your Application class (before any database operations): FirebaseDatabase.getInstance().setPersistenceEnabled(true) This caches data and queue writes while offline. When connectivity returns, Firebase syncs the local write queue automatically[18]. You can also use ref.keepSynced(true) on specific paths to keep them in sync.

Data Mapping: Follow MVVM patterns. The listener typically lives in a Repository or ViewModel, updating MutableLiveData or Flow. The View (Activity/Fragment) observes this data and updates UI components (TextViews, charts, etc). The UI (Activity/Fragment) observes viewModel.tempLiveData and binds it to views, either via data binding or manually.

Push Notifications (FCM): If you want alerts (e.g. sensor exceeds threshold), use Firebase Cloud Messaging. The Android app must be registered for FCM and can subscribe to topics. On the server side (or via a Cloud Function), send a notification to the app when certain DB events occur. For example, a Cloud Function can trigger on database writes and call firebase.messaging().send({topic: "alerts", notification: {...}}). In the app, listen for message and show a notification. (Full FCM setup is beyond scope, but see Firebase Messaging docs.)

4.6.8 Android App Architecture & UI:

Architecture: Use MVVM with Android Jetpack. For example, MainActivity hosts DashboardFragment. The repository layer handles Firebase interactions; ViewModels expose LiveData for sensor data, history, etc. Data models: e.g. data class SensorEntity(val id: String, val type: String) data class SensorReading(val timestamp: Long = 0L, val value: Double = 0.0) RecycleViews or Chart views display lists of readings.

Screens (Activities/Fragments): Typical screens might include:

Dashboard/Home: Overview of all sensors. Could use a RecyclerView of cards showing each sensor's latest value and status.

Sensor Detail: Shows a graph/history for one sensor. Implement with a chart library (e.g. MPAndroidChart) or custom view.

History: Tab or section listing timestamped readings (could be same as detail).

Settings: App settings (e.g. units, data refresh rate, Firebase project config). Implement as a Preference screen.

UI Components & Data Binding: Use XML layouts (or Jetpack Compose) with ViewBinding.

Firebase SDK Usage: In build.gradle add: implementation("com.google.firebase:firebase-database-ktx:20.0.10") implementation("com.google.firebase:firebase-auth-ktx:21.0.3") Also apply the Google Services plugin and include google-services.json in app/. Use FirebaseDatabase.getInstance() to get the root reference.

Storage & Permissions: The main permission is INTERNET. For Firebase, no special Android permissions needed beyond that. If using FCM, add com.google.firebase.MESSAGING. Use SharedPreferences or Room if you need local settings or logs, but Firebase's local cache often suffices for data.

Build Tools: Android Studio (Electric Eel or later), Kotlin 1.7+, AndroidX libraries. Use Gradle with Kotlin DSL. Min SDK ~21, target latest (e.g. 33). Use MVVM libraries (LiveData, ViewModel) and optionally Hilt/Dagger for DI.

4.6.9 Implementation Checklist:

Arduino/ESP32 Setup: Install Arduino IDE (v2.x). In Tools > Board Manager, add ESP32 support (via Espressif package)[20]. Select the correct ESP32 board (e.g. “ESP32 Dev Module”) and COM port[1].

Sensor Code: Write an Arduino sketch to initialize sensors. Test locally with Serial.println() (e.g. Serial.println(analogRead(A0))). Ensure analogRead/I2C reads correct values[3][21]. Tune any calibrations or filters.

Power/Baud Settings: Set Serial.begin(baud) (commonly 115200). If battery-powered, decide on sleep cycle. Implement esp_sleep_enable_timer_wakeup() and esp_light_sleep_start() for low-power.

Firebase Project: In Firebase console, create a Realtime Database. Note the database URL (e.g. .firebaseio.com). In Authentication, enable a sign-in method (email/password or anonymous).

Under Database > Rules, set as needed for development (e.g. temporarily true for open reads) and plan your secure rules later.

Arduino-Firebase Code: Install the Firebase_ESP_Client library (Mobizt) in Arduino IDE. In sketch, include and configure Firebase (as shown above). Test a simple write: set a value under "/test" and confirm it appears in the RTDB console

Data Schema: Decide on keys (e.g. sensor IDs). Write Arduino code to push readings (e.g. timestamp and value. Alternatively, push to a list (Firebase auto-IDs) under each sensor. Document schema for app developers.

Android Setup: In Android Studio, create a new app. Add Firebase to the project (Tools > Firebase or manually add dependencies). Include google-services.json. Enable Realtime DB in the Firebase assistant.

Android Code: Implement ViewModel and Repository that connect to Firebase. Add a ValueEventListener on each sensor path (e.g. /sensors). Map snapshots to data classes. Update LiveData accordingly.

UI Design: Build screens: main dashboard showing key readings (use RecyclerView or card views), detailed charts (e.g. MPAndroidChart for time-series), settings screen for preferences. Bind data using LiveData or DataBinding.

Test End-to-End: Run the ESP32 code; verify data appears in Firebase console. Run the Android app; verify it fetches and displays the data (with logs or breakpoints). Test offline: kill Wi-Fi on phone, ensure app shows last values and reconnects smoothly [18]. Secure & Optimize: Finalize Firebase Rules (restrict access), implement user authentication if needed. Optimize network calls (e.g. limit listeners to needed paths). Profile performance (monitor Firebase latency, Android lag).

4.6.10 Live Streaming Module Implementation:

We will use **esp32-cam** for performing this module. First we will install libraries into arduino IDE, then we will connect **wifi** with esp32-cam, then after inserting the **ip-address** of esp32-cam into app interface we will get the camera access of this module, and in this way we will enable the live streaming of patient.

4.7 Results:

4.7.1 Fall detection results

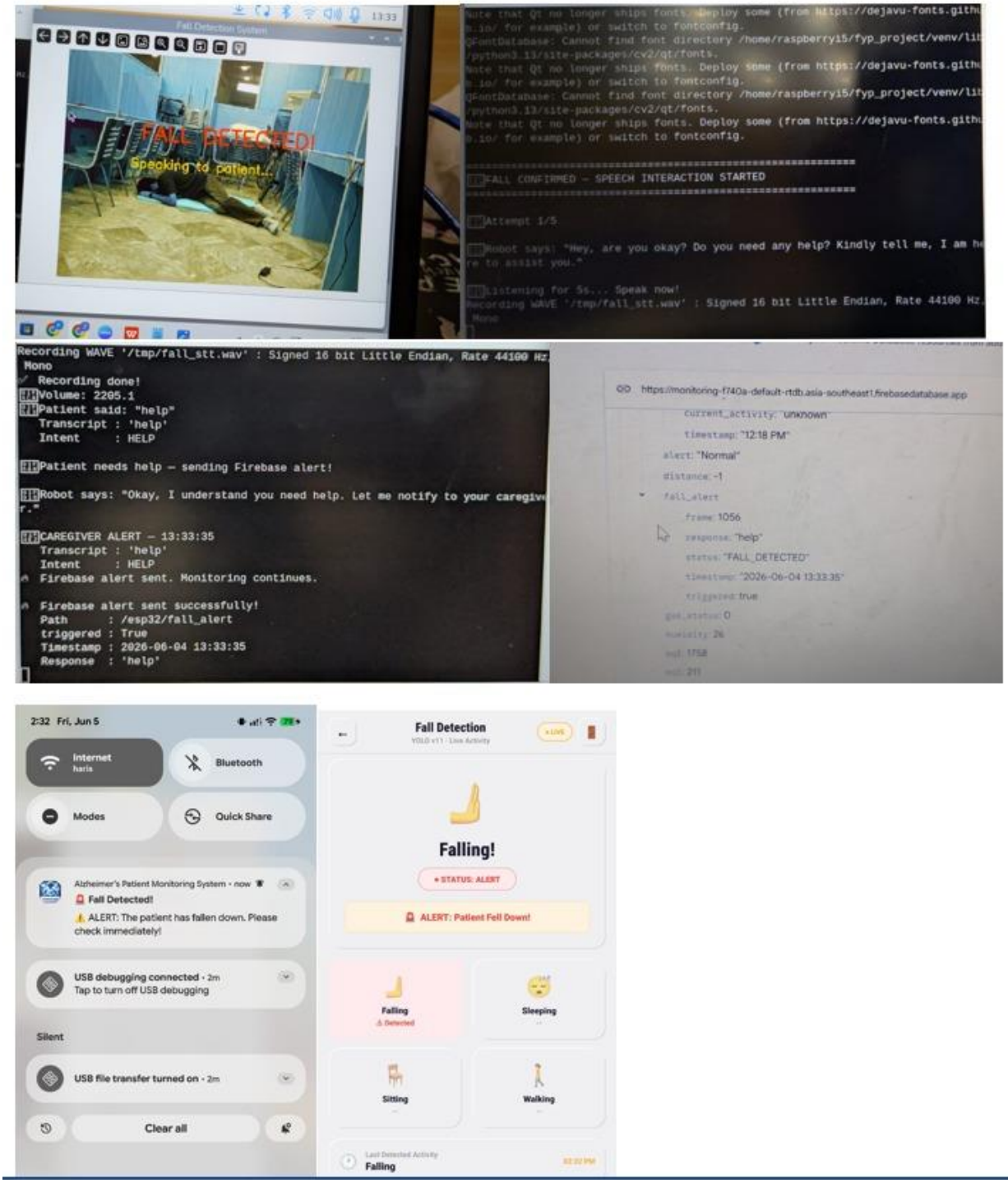


Figure 64: fall detection results

In figure 64 we have results for successfully detecting fall of patient. Fall will be detected by robot, then robot will confirm this by voice interaction with patient, if real fall occurred then alert will send to firebase cloud, from cloud we will send this alert to app, also notification will send to caregiver via app shown in figure 64.

4.7.2 Family Member Recognition Results:

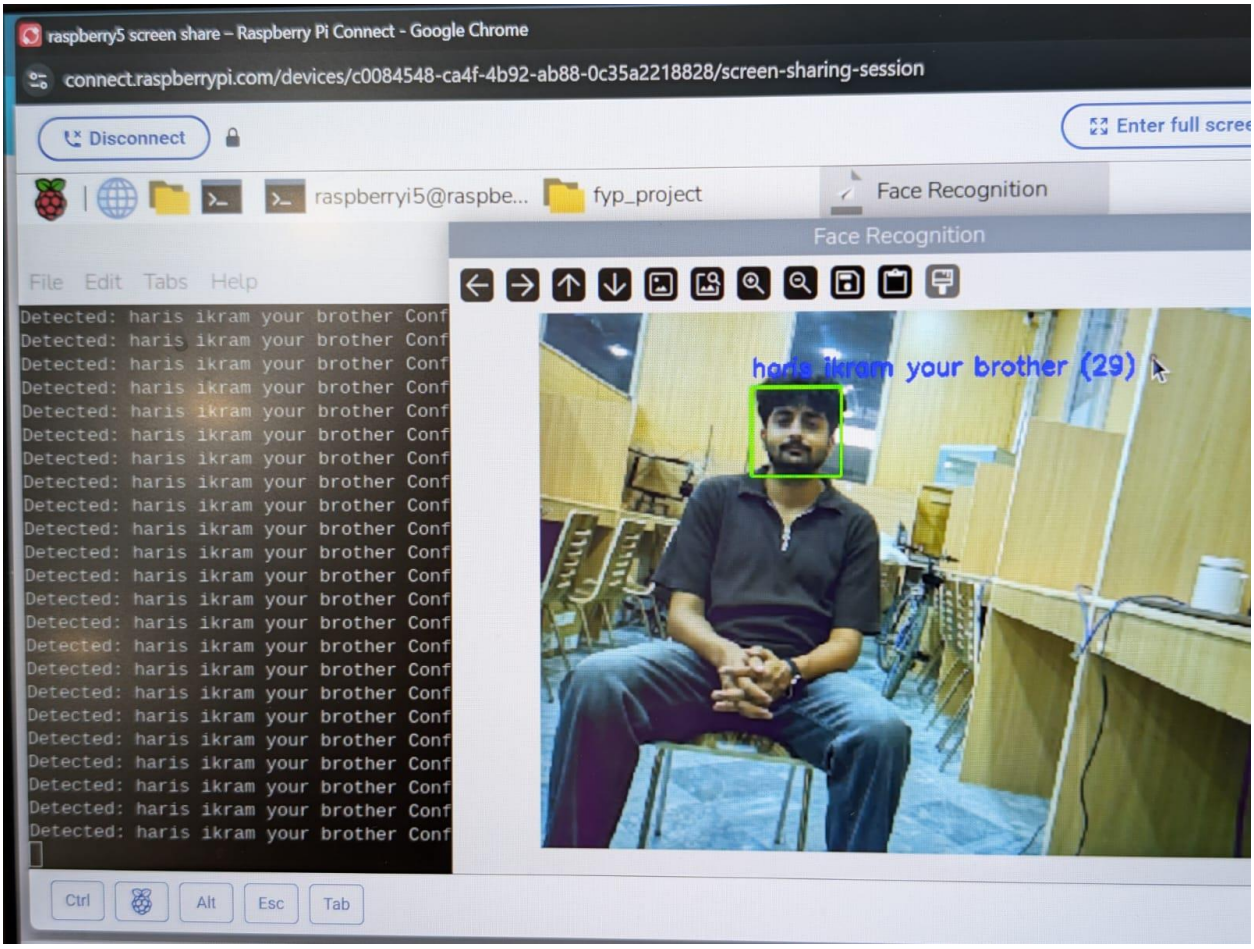


Figure 65: Family member recognition results

We success complete this module by face recognition as whenever any person will come into frame, robot will detect face, then match detected face with database saved faces and lastly tell the name and relationship labeled on that face in speakers via pi5. Shown in figure 65.

4.7.3 Results for scheduled medication alerts

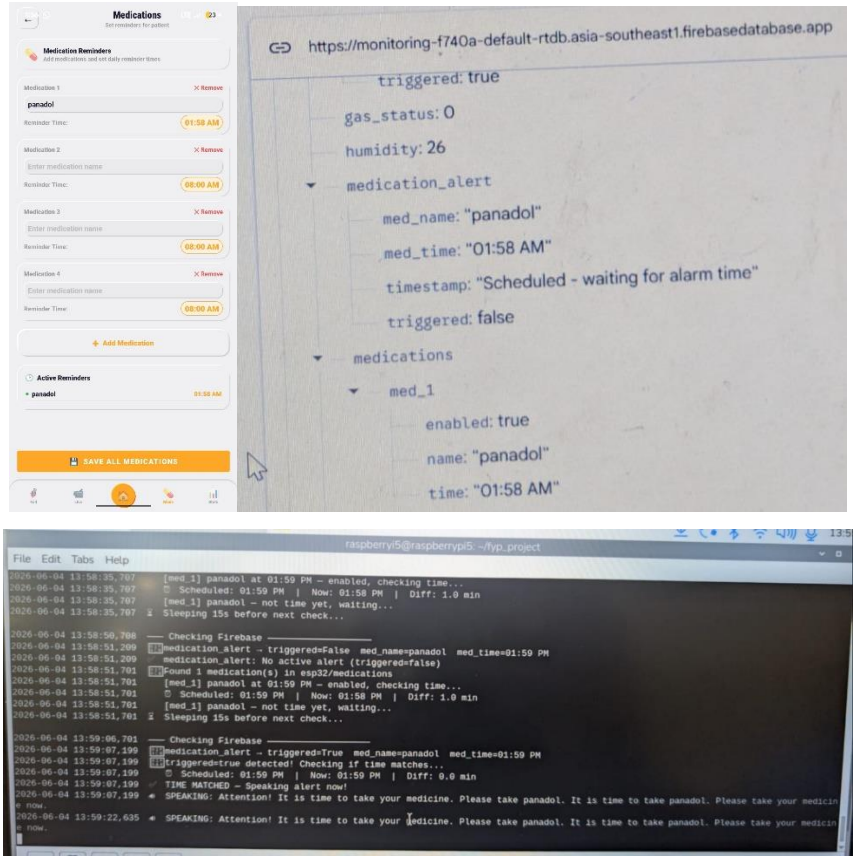


Figure 66: Scheduled Medication Alerts Module Results

The caregivers will set schedule medications from app, then send this data on firebase and from firebase pi will fetch the name and time of medication and tell to patient to take medicine on time. Shown in figure 66.

4.7.4 Environmental monitoring module results

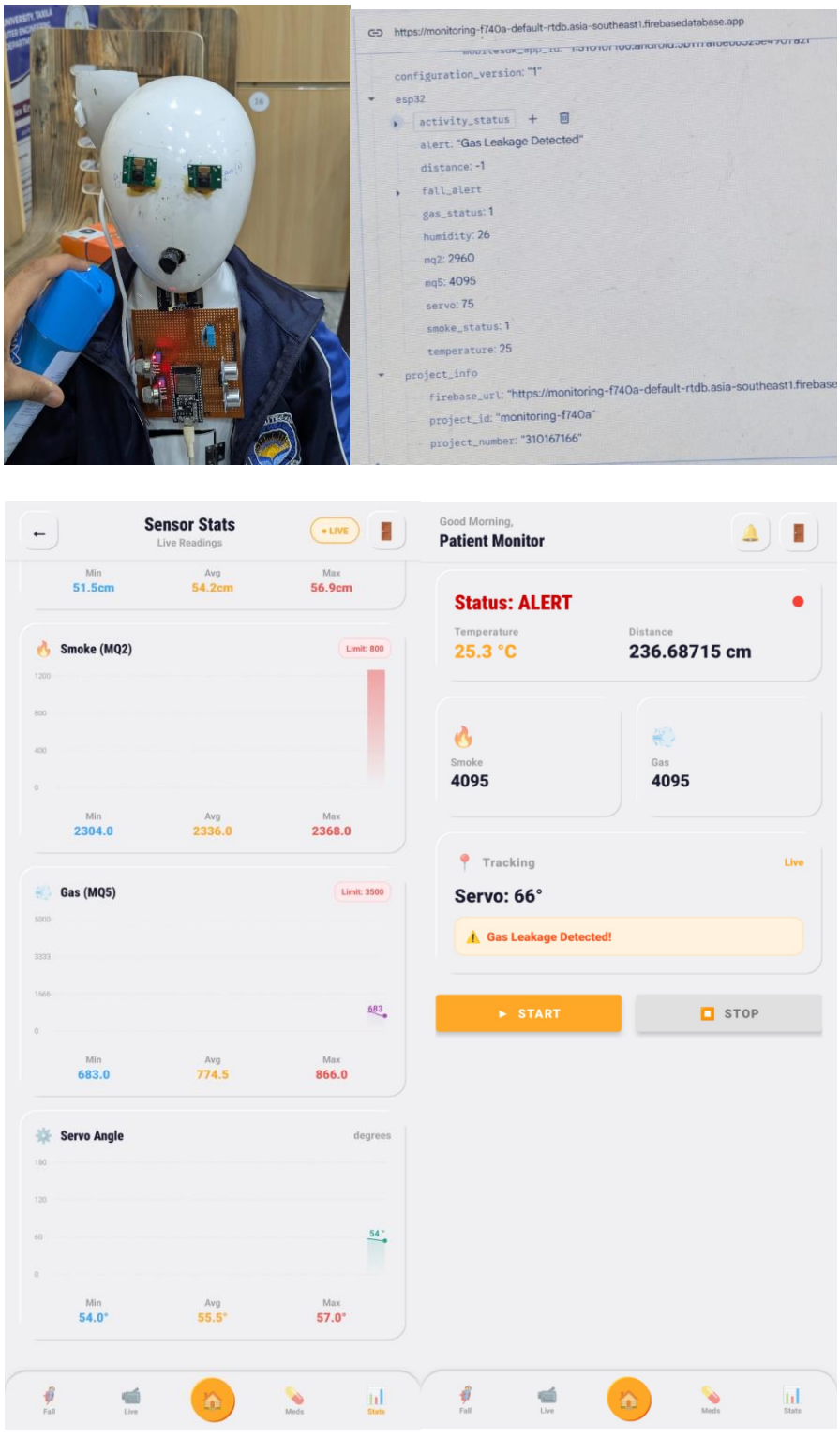


Figure 67: Environmental Monitoring Module Results

We will embed the temperature, gas and smoke sensor on robot, these will sense environment, then send the data on firebase and from firebase we will send data real time data on App. Shown in figure 67.

4.7.5 Live streaming module results

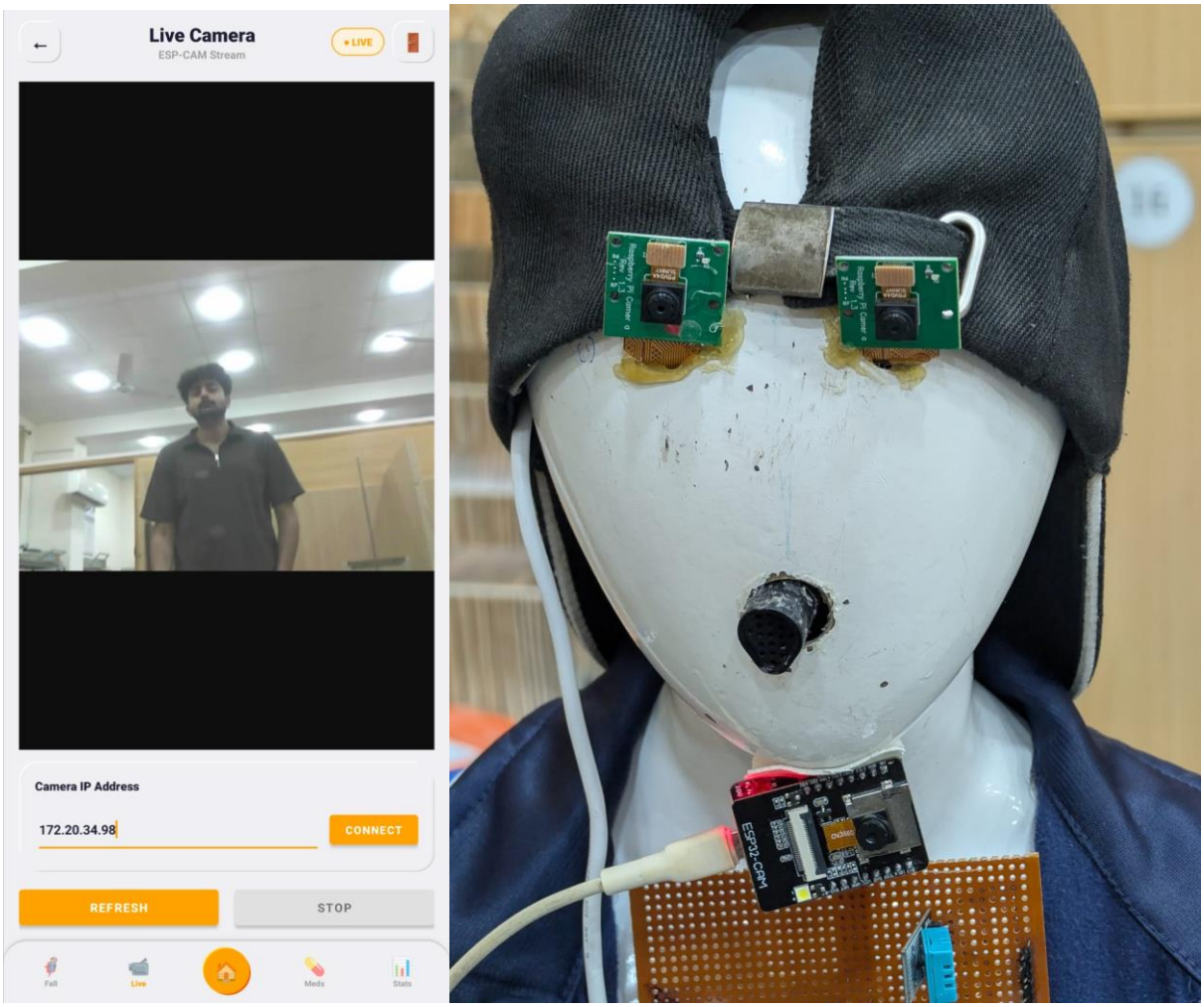


Figure 68: Live streaming results

Robot have camera and patient will use ip address of esp32-cam to get remotely observation of patient shown in figure 68.

4.7.6 App integration results

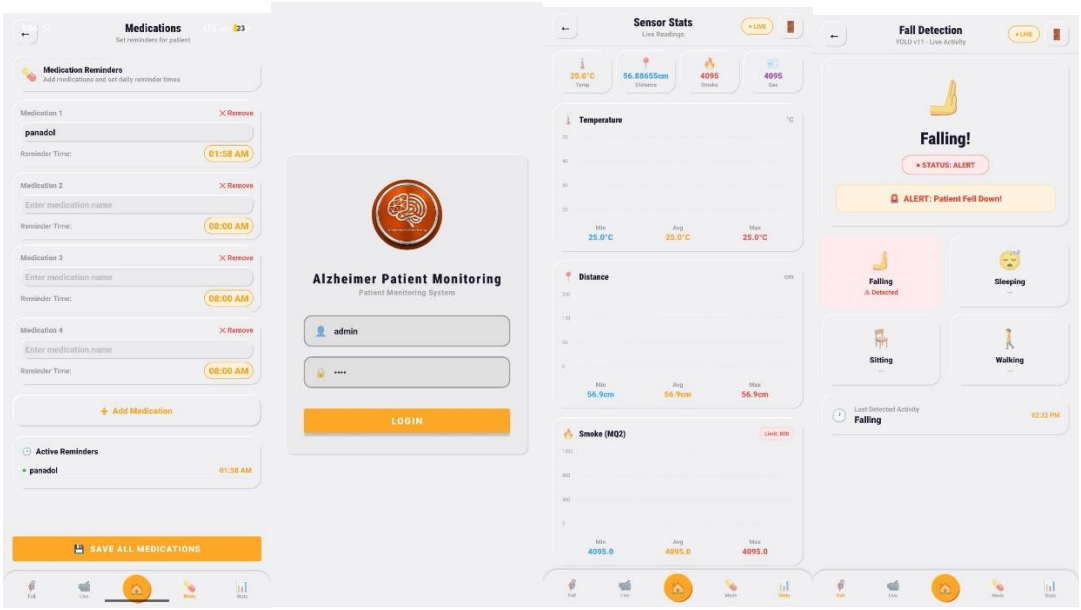


Figure 69: App results

All the data will be transferred to App and we get this integration successfully shown in figure 69.

Chapter 5
ENVIORNMENT AND SUSTAINABILITY

The Alzheimer Patient Assistance Robot operates with an emphasis upon the sustainable and efficient use of resources. This means that it should operate in a manner that is either low-power; thereby using as little electric energy while performing its duties of continuously monitoring a patient. Low-energy programming (for example, the use of very small/few onboard electric components) will allow the robot to operate only when required, thus conserving electric energy and improving battery life. The construction materials of the robot will be durable (increasing the lifespan of the robot) and lightweight (decreasing the total amount of electronic waste from the robot) for the same reason. Since there are modular components within the robot's design, maintenance or replacement of individual components can take place instead of disposing of the entire robot system when a component fails. Therefore, the robot creates an opportunity to optimize healthcare resources, as the robot decreases the reliance on continuous human monitoring of a patient. The robot also utilizes a scalable architecture, which means that future improvements can occur without doing a major redesign of the robot and that, thus, the patient's experience can continue to improve over the long term. As indicated above, this project will provide a combination of a sustainable healthcare (via patient assistance) and environmentally responsible design components.

Chapter 6

CONCLUSION

6.1 Complexities and Lessons Learned:

On this project, we encountered a considerable difficulty regarding data management, as it can be difficult to train the model and maintain all of the associated data, especially with the hardware that is involved in the training process. As a result of the need for less data to be managed, we changed our training approach to an implementation incorporating less training data. The equal and complete distribution of power to each of the hardware components is critical and the connections between the GND/GND and VCC/GND are critical to the prevention of a short circuit

6.2 Future Work:

In order to enhance our fall detection system, we will connect a microphone and speaker to Jetson Nano. Next, we will create an additional patient-following module that communicates with the microphone and speaker via fall detection results. All fall-related data will be stored in the cloud and subsequently managed from the cloud to an application. In turn, all fall data will be sent to the Jetson Nano and ESP32s and results made available to caregivers on the application.

REFERENCES

[1] Fula, V., & Moreno, P. (2024). Wrist-based fall detection: Towards generalization across datasets. *Sensors*, *24*(5), 1679. https://doi.org/10.3390/s24051679

[2] Wang, X. (2024). Egofalls: A visual-audio dataset and benchmark for fall detection using egocentric cameras. In *Proceedings of the International Conference on Pattern Recognition* (pp. 240–253). Springer Nature Switzerland.

[3] Paiva, S., & Abreu, C. (2012). Low cost GPS tracking for the elderly and Alzheimer patients. *Procedia Technology*, *5*, 793–802.

[4] AngelSense. (n.d.). *How does the AngelSense device work?* Retrieved from https://www.angelsense.com/help/#top-faqs&how-does-the-angelsense-device-work

[5] SafeWander. (n.d.). *SafeWander product information*. Retrieved from https://www.safewander.com/

[6] MetAlert. (n.d.). *GPS SmartSole product details*. Retrieved from https://metalert.shop/products/gpssmartsole

[7] Google. (n.d.). *Amazon Alexa functionalities*. Retrieved from https://www.google.com/

[8] Blue Frog Robotics. (n.d.). *Buddy robot*. Retrieved from https://www.bluefrogrobotics.com/buddy-en

[9] ElliQ. (n.d.). *ElliQ companion robot*. Retrieved from https://elliq.com/

[10] OpenCV. (n.d.). *Deep neural networks (DNN module)*. Retrieved from https://docs.opencv.org/4.x/d2/d58/tutorial_table_of_content_dnn.html

[11] Google. (n.d.). *DeepSORT*. Retrieved from https://www.google.com/

[12] SoftwareMill. (n.d.). *Comparison of PyTorch, TensorFlow, JAX, and Flax*. Retrieved from https://softwaremill.com/ml-engineer-comparison-of-pytorch-tensorflow-jax-and-flax/

[13] Arduino. (n.d.). *Upload a sketch in Arduino IDE*. Retrieved from https://support.arduino.cc/hc/en-us/articles/4733418441116-Upload-a-sketch-in-Arduino-IDE

[14] Espressif Systems. (n.d.). *ADC – Arduino ESP32 documentation*. Retrieved from https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html

[15] Espressif Systems. (n.d.). *I2C – Arduino ESP32 documentation*. Retrieved from https://docs.espressif.com/projects/arduino-esp32/en/latest/api/i2c.html

[16] Espressif Systems. (n.d.). *Serial (UART) – Arduino ESP32 documentation*. Retrieved from https://docs.espressif.com/projects/arduino-esp32/en/latest/api/serial.html

[17] Espressif Systems. (n.d.). *ESP32 series datasheet*. Retrieved from https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[18] Random Nerd Tutorials. (n.d.). *ESP32 with FreeRTOS (Arduino IDE) – Create tasks*. Retrieved from https://randomnerdtutorials.com/esp32-freertos-arduino-tasks/

[19] Espressif Systems. (n.d.). *Current consumption measurement of modules – ESP32*. Retrieved from https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/current-consumption-measurement-modules.html

[20] GitHub. (n.d.). *EspExceptionDecoder for ESP8266 and ESP32*. Retrieved from https://github.com/me-no-dev/espexceptiondecoder

[21] GitHub. (n.d.). *Firebase-ESP32 library documentation*. Retrieved from https://github.com/mobizt/Firebase-ESP32/blob/master/src/README.md

[22] Firebase. (n.d.). *Read and write data on Android – Realtime Database*. Retrieved from https://firebase.google.com/docs/database/android/read-and-write